GRP Project Catalogue 2026-2027

Prof. Dave Towey

School of Computer Science

October 13, 2026

Contents

Introduction **8**
Project Allocation Key Points . . . 8
Project Preferences . . . 8
Project Bidding . . . 8
Project Awarding . . . 10

Projects **11**
[P01] Mortgage and Income Tax Joint Calculator: Maximizing After-Tax Income . . . 11
Requirements . . . 11
Background . . . 11
Outline . . . 11
Tasks & Deliverables . . . 12
[P02] One Campus, A Thousand Stories: An Augmented Reality Campus Journey . . . 14
Requirements . . . 14
Background . . . 14
Outline . . . 14
Tasks & Deliverables . . . 16
Additional Information . . . 17
References . . . 17
[P03] SignBridge: An AI-Powered Two-Way Sign Language Communication System . . . 18
Requirements . . . 18
Background . . . 18
Outline . . . 19
Tasks & Deliverables . . . 19
Additional Information . . . 20
[P04] AI-driven personalized learning paths . . . 22
Requirements . . . 22
Background . . . 22
Outline . . . 22
Tasks & Deliverables . . . 22
[P05] ShotSense AI: Can AI Coach Your Basketball Shot? . . . 24
Requirements . . . 24
Background . . . 24
Outline . . . 24
Tasks & Deliverables . . . 24
Additional Information . . . 25

[P06] AI Assisted Website Accessibility Testing and Repair . . . 26
Requirements . . . 26
Background . . . 26
Outline . . . 26
Tasks & Deliverables . . . 26
Additional Information . . . 27
[P07] A Software Tool for Qualitative Spatial and Temporal Reasoning . . . 28
Requirements . . . 28
Background . . . 28
Outline . . . 28
Tasks & Deliverables . . . 28
[P08] Text2Visual: An Intelligent System for Transforming Textual Content into Visual Narratives . . . 30
Requirements . . . 30
Background . . . 30
Outline . . . 30
Tasks & Deliverables . . . 31
Additional Information . . . 32
[P09] Creating a Sustainable Workflow for Virtual Production (VP) Filmmaking . . . 33
Requirements . . . 33
Background . . . 34
Outline . . . 34
Tasks & Deliverables . . . 35
References . . . 35
[P10] ErgoAssist: A Specialised LLM for Anthropometric Seating Design . . . 36
Requirements . . . 36
Background . . . 36
Outline . . . 36
Tasks & Deliverables . . . 37
Additional Information . . . 38
[P11] Level Up: Serious Games for 21st-Century Employability Skills . . . 39
Requirements . . . 39
Background . . . 39
Outline . . . 39
Tasks & Deliverables . . . 40
Additional Information . . . 41
[P12] CampusWorld: A Smart Campus World Model Sandbox . . . 42
Requirements . . . 42

Background . . . 42
Outline . . . 42
Tasks & Deliverables . . . 43
Additional Information . . . 43
[P13] AI-Powered Personalized Learning System for Secondary School Students . . . 44
Requirements . . . 44
Background . . . 44
Outline . . . 44
Tasks & Deliverables . . . 45
Additional Information . . . 45
[P14] Software Tool for Fairness in Machine Learning . . . 46
[P15] Interactive Multi-UAV Viewpoint and Path Visualization Tool . . . 47
Requirements . . . 47
Background . . . 47
Outline . . . 47
Tasks & Deliverables . . . 48
Additional Information . . . 50
[P16] PhishGuard: An Explainable Machine-Learning Phishing Detection and Security Awareness Platform . . . 51
Requirements . . . 51
Background . . . 51
Outline . . . 52
Tasks & Deliverables . . . 52
Additional Information . . . 55
References . . . 55
[P17] An Intelligent Auto-Clamping Car Mount with Pressure-Sensitive Gesture Recognition 57
Requirements . . . 57
Background . . . 57
Outline . . . 57
Tasks & Deliverables . . . 58
Additional Information . . . 59
[P18] An Automated Testing Framework for Conversational Agentic Systems . . . 61
Requirements . . . 61
Background . . . 61
Outline . . . 61
Tasks & Deliverables . . . 62
Additional Information . . . 62
References . . . 62

[P19] SmartCareer: A Personalized Employment Recommendation System with a Multi-Dimensional Job Quality Evaluation Model . . . 64
Requirements . . . 64
Background . . . 64
Outline . . . 64
Tasks & Deliverables . . . 65
Additional Information . . . 66
[P20] A Large Language Model-enabled Learning Pipeline with Genetic Optimizers for Generating Inorganic Materials as Texts . . . 67
Requirements . . . 67
Background . . . 67
Outline . . . 68
Tasks & Deliverables . . . 68
Additional Information . . . 69
[P21] AI-Driven Post-Investment Quarterly Report Analysis for Private Equity Portfolios . . . 70
Requirements . . . 70
Background . . . 70
Outline . . . 70
Tasks & Deliverables . . . 71
Additional Information . . . 71
[P22] AI-driven Magnetic Nano-grain Analysis and Evaluation System . . . 72
Requirements . . . 72
Background . . . 72
Outline . . . 72
Tasks & Deliverables . . . 72
[P23] Automated Assessment Platform with Teacher-Confirmed Multimodal Feedback . . . 73
Requirements . . . 73
Background . . . 73
Outline . . . 73
Tasks & Deliverables . . . 74
[P24] FitTwin: An AI Digital Twin for Personalized Strength Training and Recovery Optimization 75
Requirements . . . 75
Background . . . 75
Outline . . . 76
Tasks & Deliverables . . . 77
Additional Information . . . 78
[P25] AI Wardrobe Studio . . . 79
Requirements . . . 79

Background . . . 79
Outline . . . 79
Tasks & Deliverables . . . 80
Additional Information . . . 81
[P26] CodeLingo: Learning to Program Through Short Daily Games . . . 82
Requirements . . . 82
Background . . . 82
Outline . . . 82
Tasks & Deliverables . . . 82
Additional Information . . . 83
[P27] IndustrialSense: Software Platform for an Optical Sensing and Process Monitoring Prototype . . . 84
Requirements . . . 84
Background . . . 84
Outline . . . 84
Tasks & Deliverables . . . 85
[P28] Developing an Online Learning Platform for Competitive Software Testing and Bug Fixing 86
Requirements . . . 86
Background . . . 86
Outline . . . 87
Tasks & Deliverables . . . 88
Additional Information . . . 89
References . . . 89
[P29] Real-Time Surgical Video AI Assistant Development . . . 90
Requirements . . . 90
Background . . . 90
Outline . . . 91
Tasks & Deliverables . . . 91
Additional Information . . . 93
[P30] “From Prompt to Film” — An AI-Empowered Micro-Film Studio . . . 95
Requirements . . . 95
Background . . . 95
Outline . . . 95
Tasks & Deliverables . . . 96
Additional Information . . . 97
[P31] Agentic Image Enhancement System for Challenging Visual Conditions . . . 98
Requirements . . . 98
Background . . . 98

Outline . . . 98
Tasks & Deliverables . . . 98
Additional Information . . . 99
[P32] RapidAid: Dispatching and Routing Medical Responders in an 80,000-Seat Stadium . . . 100
Requirements . . . 100
Background . . . 100
Outline . . . 100
Tasks & Deliverables . . . 101
Additional Information . . . 101
[P33] LLM-Empowered User Profiling for Recommendation . . . 102
Requirements . . . 102
Background . . . 102
Outline . . . 102
Tasks & Deliverables . . . 103
Additional Information . . . 103

Introduction

This catalogue of project briefs should be read in conjunction with the GRP handbook, available through Moodle.

Project Allocation Key Points

As noted in the handbook:

- Each team must work on a different project to every other team — no two teams may work on the same (or even similar) projects.
- The projects may not be very clearly described.
- Initially, there are no supervisors for each project. This means that there is nobody to approach for additional information (at this stage).
- Projects that are available initially, may change, or even disappear.
- Each team should go through all the projects and identify up to three that they are most interested in doing.
- Each team may then submit up to three "bids," one for each of the identified preferred projects.
- There is no guarantee that a team will get its preferred project. There is no guarantee that a team will even get a project that they want!
- Each team may compete with other teams to be awarded their preferred project(s), initially through the bids.
- There may (initially) be more teams than projects. In this case, those teams not assigned one of the initially offered projects will be assigned a new project at the end of the bidding process.

Project Preferences

Soon after you have been put into your teams, you should discuss which projects you would most like to do. You should decide on the top three choices. (Note that it is possible that you may end up with a different project to these choices.)

When you have identified your top three projects, your team should prepare your "bids" for each of your choices.

Project Bidding

The "bidding" phase involves each team preparing a package (called a bid) that they use to try to convince the project proposer to award the project to them. Each team may submit up to three bids

(ranked in order of preference). The submission link and detailed explanation will be in Moodle. All bids must be submitted by the deadline: No late submissions will be accepted.

Each bid file should be a single archive file, in .zip format, containing all relevant materials. The bid file should be named as follows:

```
<TeamID>.<PreferenceNumber>.<ProjectID>.zip
```

where TeamID and ProjectID are the team and project identifiers (e.g., Team202607 and P11), and PreferenceNumber is 1, 2, or 3 (indicating 1st, 2nd, or 3rd preference — where 1 is the most preferred).

For example, if Team202607 wanted to do project P11 as their first preference, their submission file would be named:

```
Team202607.1.P11.zip
```

Please note that, sometimes, due to restrictions on filetypes accepted by the submission tool, it may be necessary to add an extra extension to the filename. (When this is necessary, it will be clearly stated at the submission point.) For example, it may be necessary to rename the file above to be:

```
Team202607.1.P11.zip.pptx
```

The individual bid should contain all the files and materials that the team will use to attempt to convince the project proposer to award the project to them. There are many possible things to include in the bid, for example:

- A letter outlining why the team is a good candidate for completing the project
- Details of relevant experience or expertise. Some teams may like to include CVs or resumés for their team members
- Some proposed ideas related to the project
- An estimated timeline or schedule for how the team would complete the project on time

Not all bids would need to include all of the above items, and obviously there are many other things that could also be included. Your team should spend time considering how best to convince the project proposer to award you the project.

Note that the file upload size limits may mean that you should take steps to keep your bid file as small as possible. You should also take steps to make sure that you do not encounter last-minute network issues: It is the responsibility of each team to ensure that their bid is submitted. It has been noticed in the past that network traffic immediately before the deadline can cause last-minute submissions to not be accepted: Aim to submit with sufficient time before the deadline.

Project Awarding

Once the deadline for bidding has past, all the bids will be examined, and some projects will be awarded.

If any project has only one bid submitted, and that bid was as a first preference, then the bidding team will be awarded that project.

If any project has more than one bid as first preference, then the relevant bids will be examined and one chosen.

Once all the first preference bids have been processed, the second preferences will be examined in a similar manner, followed by the third preferences.

The projects not yet awarded at the end of the preferences processing stage will be then examined and matched to the remaining teams. Details of if and how this will be done will be explained in the class sessions.

At the end of this phase, each team will have a project.

Projects

[P01] Mortgage and Income Tax Joint Calculator: Maximizing After-Tax Income

Requirements

- Proficiency in programming languages such as Python.
- At least ONE student in the group should be able to search, read, and understand relevant regulations in Chinese.
- Basic ability in optimization algorithms or numerical computation.
- Able to develop a simple software interface.
- General knowledge of data processing, testing, and software development.

Background

In real life, personal income, personal income tax, housing provident fund, and mortgage payments are not independent of each other. Salary and wages affect personal income tax and housing provident fund contributions. Housing provident fund contributions further affect mortgage eligibility and repayment methods. The combination of commercial loans and housing provident fund loans, loan terms, and repayment methods can significantly affect monthly cash flow and long-term after-tax income.

Therefore, a simple ”income tax calculator” or ”mortgage calculator” is often not enough to support real user decisions. What users truly care about is: under legal and compliant conditions, how to adjust salary structure, housing provident fund contributions, total mortgage amount, loan term, and the ratio between commercial loans and housing provident fund loans, so that their after-tax income minus mortgage payments is as high as possible.

This project aims to develop a ”Mortgage and Income Tax Joint Calculator.” The system should allow users to input their salary and salary composition, calculate after-tax income and mortgage payments based on current laws and regulations, and optimize user-adjustable parameters to maximize take-home income. The system can be a command-line program, a desktop application, or a web application.

Outline

This project aims to implement a mortgage and income tax joint calculator and study how different parameter combinations affect after-tax income. Providing further visualization analysis and optimiza-

tion suggestions is a bonus.

Consider a user who has a certain salary and wage income. The salary may consist of basic salary, performance pay, bonuses, allowances, housing provident fund, social insurance, and other components. The user wants to purchase a house and therefore needs to apply for a mortgage. The mortgage may include commercial loans and housing provident fund loans, which have different interest rates, credit limits, and repayment rules. The user can adjust parameters within a certain range, such as total loan amount, loan term, the ratio between commercial loans and housing provident fund loans, and repayment methods.

The system needs to complete the following process:

1. The user inputs personal salary information, salary composition, housing provident fund contribution ratio, social insurance contribution, and candidate mortgage parameters, etc.
2. The system calculates monthly after-tax income, monthly mortgage payment, and annual cash flow based on current income tax rules, housing provident fund rules, and mortgage rules.
3. Under legal, compliant, and user-given constraints, the system searches for or optimizes adjustable parameters, such as loan term, loan combination, and housing provident fund usage.
4. The system outputs recommended plans, including after-tax income, mortgage payment, annual take-home income, total interest, and optimal parameter combinations.
5. If possible, the system provides charts to show differences between different plans and help users understand the optimization results.

A typical usage flow is as follows:

1. The user inputs pre-tax salary, bonus, allowance, housing provident fund contribution ratio, social insurance base, and other information.
2. The user inputs total house price, down payment ratio, acceptable loan term range, and whether to use a housing provident fund loan.
3. The system calculates monthly payments, total interest, after-tax income, and net take-home income under different loan combinations and terms.
4. The system provides a recommended plan that maximizes after-tax income (interest/inflation considered) and shows a comparison with the default plan or the user's original plan.

Tasks & Deliverables

In this group project, students are asked to:

- Design a data model that can represent salary composition, income tax, housing provident fund, and mortgage parameters.
- Implement an after-tax income calculation module and a mortgage payment calculation module, including common repayment methods such as equal principal and interest and equal principal.
- Implement combined calculation of commercial loans and housing provident fund loans.
- Implement a parameter optimization module to maximize after-tax income or net take-home income under user constraints.
- Compare the differences in results under different salary structures, loan terms, and loan combinations.
- Develop a complete software system that wraps up calculation, optimization, and result presentation. This software should be compatible with at least one mobile operating system and one computer operating system.
- Provide a user-friendly interface, such as command-line interaction, a desktop interface, or a web interface.
- Present your work.
- Work in a group, collaborate, and learn from each other.

[P02] One Campus, A Thousand Stories: An Augmented Reality Campus Journey

Requirements

This project involves mobile application development, augmented reality (AR), game design, and human-computer interaction. The selected team will develop proficiency in a suitable development environment, such as Unity with AR Foundation. Interface design, audio recording, and database management will also be involved. An interest in interactive storytelling and campus culture would be valuable.

The project will require campus information, visual material, and voluntarily contributed stories. NFC tags may provide checkpoint access, with QR codes as an alternative. Relevant stakeholders may include campus visitors, current students, graduating students, alumni, and staff responsible for campus tours or university history. The team will identify suitable locations and agree the scope with the project supervisor.

Background

A university campus holds more than buildings and facilities. It also holds memories of first lectures, friendships, achievements, and graduation. A campus tour offers an opportunity to connect these personal experiences with the places where they happened. Visitors could discover university life through the voices of its community, while graduating students could leave something meaningful for future cohorts.

Research on AR storytelling offers useful design perspectives. Raeburn, Welton, and Tokarchuk (2022) developed and evaluated mobile experiences combining outdoor exploration with AR narratives. Hirsch et al. (2022) compared informative and interactive AR stories at historical places. Together, these studies motivate attention to how digital stories relate to physical surroundings and how visitors participate in them.

The tourism application design presented by FUTU (2026) combines a character guide with visual stories and personal journey records. These features provide a practical reference for linking exploration with storytelling. The proposed project will adapt these ideas to the University of Nottingham Ningbo China (UNNC), connecting campus locations with stories from visitors, students, and graduates.

Outline

The project aims to design, develop, and evaluate a mobile AR application for self-guided campus exploration and shared storytelling. The proposed initial scope is four to six story points at selected campus landmarks. Visitors will choose which locations to visit and explore them at their own pace.

Each story point will provide direct access to campus information, AR content, and personal memories. The central experience will be to discover a place, explore its stories, and optionally leave a new contribution.

Key research questions (RQs):

1. How can AR interactions and self-guided storytelling support engaging campus exploration while keeping attention on the surrounding places?
2. How do visitors interpret campus locations through personal voice stories and messages from graduates, and how do they describe their connection to those places?
3. What prompts and interaction choices encourage visitors and graduating students to contribute meaningful stories while keeping control over how they are shared?
4. How can one application support both visitors discovering the campus and graduating students recording memories of familiar places?

Proposed experience

- **Explore UNNC Campus**: Visitors, prospective students, families, and new students will select places of interest on an illustrated campus map. Each location will have a short preview to help visitors decide where to go. An original character guide will introduce the campus and offer optional directions. Visitors may skip locations, change their location, or finish their visit at any point. At a story point, the visitor will tap an NFC tag or scan a QR code to open its content. The visitor can then view campus information, browse personal stories, or open an AR scene. For example, a virtual memory box could appear beside a recognised marker. Selecting it would reveal messages and recordings associated with that location. This is a proposed interaction rather than an existing campus installation. Visitors will be able to listen to recordings or read messages, with published contributions organised by location and graduation year where relevant. A proposed recording limit of 30-60 seconds will keep stories focused. Audio will play only after selection, and text alternatives will be available. Visitors may leave a contribution at a story point or continue without doing so.
- **Leave a Memory**: Graduating students will be able to open a dedicated contribution mode directly at selected locations. Prompts could include a memorable moment at that place or advice for a future student/junior. Contributors will preview their recording or message, select a display name, and choose whether to submit it for public viewing. Submitted contributions will enter a review queue before publication. The initial collection will be gathered from willing contributors, including alumni where available.
- **Campus virtual stamp collection**: Visitors may collect a virtual illustrated stamp after tapping or scanning a story point. These optional stamps will record visited locations, and visitors will

be able to save favourite stories for later browsing. Story access and contribution options will remain available regardless of the number of stamps collected. Chinese and English interface text will support different visitor groups. A conventional map and story view will also be available. The first prototype will target one mobile platform, with further platform support considered as an extension.

Tasks & Deliverables

The selected students will work as a software engineering team and complete an iterative development process during the GRP period. Requirements gathering, feasibility assessment, and project scoping will inform a final proposal. The team will select an appropriate development process and maintain planning and progress records. Expected tasks and deliverables include:

1. **Requirements specification and project plan**: Identify stakeholder needs through suitable interviews or discussions. Define user roles, story-point interactions, content requirements, and acceptance criteria. To agree milestones and responsibilities with supervisor.
2. **Interaction and visual design**: Produce user journeys, wireframes, an illustrated map, and story-point storyboards. Develop the visitor, contributor, and content-review workflows through successive prototypes.
3. **Working mobile prototype**: Implement map browsing, story-point access, AR interactions, and the campus stamp collection. Include story recording, playback, and browsing by location and graduation year.
4. **Content collection and management interface**: Prepare approved campus information and a small collection of contributed stories. Provide tools to edit story points, review submissions, publish messages, and handle reporting and deletion.
5. **Verification and evaluation report**: Test complete workflows and recovery from common failures. Conduct a formative user evaluation and document changes made in response to findings.
6. **Final demonstration and handover**: Deliver source code, technical documentation, installation instructions, and user guidance. Include content-management guidance, test reports, and recommendations for continued development.

Proposed evaluation

The evaluation will involve student/staff representing the visitor and contributor roles. Visitors will choose locations from the map, access their content, and save a favourite story. Graduating students will create, preview, and submit a contribution. Observations will capture navigation difficulties, interaction errors, task completion, and time taken. Short questionnaires and interviews will address

enjoyment, story relevance, perceived connection to campus, and willingness to contribute. Feedback from the two roles will be considered separately.

These activities will address the research questions through observed use and participant accounts. A small formative study will guide design improvements rather than establish lasting changes in belonging. If time and recruitment permit, an additional comparison could present the same locations and stories with and without AR. This would help assess the contribution of AR presentation. The study protocol, ethics and recruitment arrangements will be agreed and approved before data collection.

Additional Information

Candidate locations include the Trent Building, the library, the bridges, the iconic animal (e.g. MiNuo) and suitable viewpoints near Nottingham Lake, identified on the official UNNC campus map. Final story-point selection will consider access, walking conditions, and permission for temporary markers. Each stop should provide space to interact with the application while stationary.

Potential extensions include themed walks organised around campus history or student life, graduation time capsules that open on a chosen date, and personalised digital postcards. Historical photographs could support then-and-now activities. A searchable campus archive or conversational guide could follow the core prototype.

Official campus information and personal memories will be labelled separately. Contributors will control whether to submit material publicly and will have a way to delete it. The team will agree content ownership, storage, and review arrangements. A content owner and maintenance plan will be needed for continued use after the student project.

The project could support further work in AR interaction, digital storytelling, and campus visitor experiences. Academic dissemination would depend on the completed evaluation and its findings.

References

1. Raeburn, G., Welton, M., and Tokarchuk, L. (2022). Developing a play-anywhere handheld AR storytelling app using remote data collection. *Frontiers in Computer Science*, 4, 927177.

2. Hirsch, L., Welsch, R., Rossmy, B., and Butz, A. (2022). Embedded AR Storytelling Supports Active Indexing at Historical Places. *Proceedings of the Sixteenth International Conference on Tangible, Embedded, and Embodied Interaction*.

3. University of Nottingham Ningbo China. Campus map.

4. FUTU (2026). AI and XR tourism application.

[P03] SignBridge: An AI-Powered Two-Way Sign Language Communication System

Requirements

This project aims to develop an AI-powered two-way sign language communication system that can support communication between sign-language users and non-signing users. It will integrate visual sign analysis, speech/language processing and software system development to provide an end-to-end communication experience. Specifically, the team is required to have expertise, or a strong willingness to develop expertise, in computer vision for video/sign analysis, natural language and speech processing for text/speech input and output, and software engineering for system integration and user-interface development. The team should be proficient in programming languages suitable for AI/ML development, such as Python, and familiar with relevant libraries and frameworks such as OpenCV, MediaPipe, TensorFlow or PyTorch. Experience with application development, pretrained or open-source AI models, APIs, version control and software testing would be useful. Team members should also be willing to collect, annotate and manage a small research dataset responsibly, following applicable consent, privacy and research-ethics requirements.

Background

Sign-language users may face communication barriers when interacting with people who do not understand sign language, particularly when an interpreter is not available. A practical communication tool should support both directions of a conversation: a signer should be able to communicate with a non-signing person, and the non-signing person should be able to respond in a form that can be understood by the signer.

Recent advances in computer vision, hand and human-pose analysis, speech recognition, natural language processing and generative AI make it increasingly feasible to develop assistive systems that connect visual signing with text and speech. However, many existing demonstrations focus on only one communication direction or on an isolated AI model rather than a complete, usable software system.

This project aims to build an integrated two-way communication platform. In one direction, camera input from a signer will be analysed and converted into text and, where appropriate, synthesised speech. In the other direction, speech or typed text from a non-signing user will be converted into a sign-language representation and presented visually, for example through an animated avatar, pose/skeleton visualisation, sign clips or another suitable approach. The project will also create a small-scale mobile sign-language dataset recorded under realistic usage conditions to support development and evaluation of the sign-recognition component.

Outline

The team will design and implement SignBridge as an integrated software system. The system should provide a simple conversational experience in which users can switch naturally between the two communication directions. The team will determine the most appropriate implementation platform, system architecture and user interface based on requirements, technical feasibility and intended usage scenarios.

The proposed system should include the following main components:

• Sign-to-text/speech: capture signing through the device camera, process the visual input using an appropriate recognition pipeline, and present the recognised content as text with optional text-to-speech output.

• Speech/text-to-sign: accept spoken or typed input, convert speech to text where necessary, process the input, and present the corresponding sign-language output using an appropriate sign-generation or sign-visualisation method.

• Conversation interface: provide clear turn-taking, recognised/generated output, replay or retry functions, and a simple conversation history so that both users can follow the exchange.

• Mobile sign-language dataset: design a small-scale data-collection protocol, collect and annotate representative samples under realistic mobile usage conditions, and use the data to develop and evaluate the recognition component.

• Modular system and AI integration: integrate the recognition, speech/language and sign-visualisation components through well-defined software modules, services or APIs so that they can be independently developed, tested and improved.

The team is not expected to develop every AI model from scratch. Existing datasets, pretrained models, open-source libraries and suitable APIs may be used where appropriate. The emphasis is on designing, integrating, testing and evaluating a robust end-to-end software system, while still carrying out meaningful implementation and evaluation of the AI components.

Tasks & Deliverables

The prospective team will be expected to complete the following tasks and deliverables:

1. Requirements analysis and project planning: identify representative communication scenarios, define functional and non-functional requirements, and produce a realistic development plan.

2. Technical investigation and system design: review suitable approaches for sign recognition, speech-to-text, text-to-speech and sign visualisation/generation; design the overall architecture, data flow, interfaces and technology stack.

3. Dataset creation: define a manageable data-collection and annotation protocol, obtain any required approvals/consent before data collection, create a small-scale mobile sign-language dataset, and document its structure and intended use.

4. Sign-recognition component: implement a baseline recognition pipeline using suitable computer-vision/AI techniques and evaluate it using appropriate train/validation/test procedures and performance metrics.

5. Speech/text and sign-output component: integrate speech recognition and text input, then implement a practical method for converting the message into visual sign-language output. Existing models or assets may be adapted or integrated where appropriate.

6. System implementation: implement the chosen application/platform and user interface, together with the necessary input/output functions, model/API integration and conversation-management features.

7. System integration and testing: integrate all components into a complete two-way workflow and conduct functional, usability, robustness and performance testing across representative usage scenarios and the chosen implementation environment.

8. Evaluation and refinement: analyse recognition quality, response time and user experience; identify limitations; and improve the system based on testing results.

9. Final deliverables: a working prototype, source code and technical documentation, the documented small-scale dataset (subject to consent/privacy restrictions), test/evaluation results, a project report, and a live or recorded demonstration.

Additional Information

The exact sign language, vocabulary size and form of sign visualisation/generation can be agreed with the supervisor after the team has investigated available resources and technical feasibility. The initial system may focus on a constrained vocabulary or selected communication scenarios; a reliable end-to-end prototype is more important than attempting unrestricted continuous sign-language translation.

Data collection involving human participants must follow relevant University procedures for consent, privacy, secure storage and research ethics. The team should avoid collecting unnecessary personal information and clearly document how the dataset is collected, stored and used.

This project has strong potential for academic dissemination in sign-language recognition and translation, multimodal AI, assistive technology, human-computer interaction and accessible computing. For publication, rigorous empirical evaluation would be important, including recognition/translation performance, system usability and evidence from realistic communication scenarios. The small-scale

dataset may also support research dissemination if it is collected, documented and evaluated appropriately.

[P04] AI-driven personalized learning paths

Requirements

The project involves Artificial Intelligence, Machine Learning, Natural Language Processing, Data Analytics, and System Design. The team should have proficiency in all the technologies as well as current tools and techniques for them. Additionally, they have proficiency in AI programming using Python, and frontend and backend design and development tools and techniques. The team should also have hands-on experience with Large Language Models and their uses.

Background

AI-driven personalized learning pathways are expected to have a profound impact. By bridging assessment gaps, boosting student engagement, and supporting differentiated instruction, the project aims to foster educational equity and excellence. It empowers learners to take charge of their own educational journeys, accelerates remedial support for students facing difficulties, and enables gifted students to pursue advanced learning without constraints. For teachers, the project provides in-depth insights into student progress and automates routine administrative tasks, thereby freeing up more time for meaningful interaction and mentorship. Ultimately, the AI-driven personalized learning pathways project represents a transformative approach to education; it harnesses the potential of AI not only to enhance efficiency and scalability but also to build a learning ecosystem that is more equitable, motivating, and interactive.

Outline

As educational environments become increasingly diverse and student-centered, the limitations of traditional teaching models have driven a shift toward personalized learning experiences that acknowledge and respect each student's unique needs, talents, and aspirations. The AI-driven personalized learning path project introduces an innovative solution that leverages state-of-the-art AI technology to revolutionize educational delivery, engagement, and outcome assessment. By integrating machine learning, natural language processing (NLP), and data analytics, the project aims to build an adaptive, scalable platform designed to optimize learning for students across various educational stages.

Tasks & Deliverables

At the core of the project's approach is the continuous, comprehensive analysis of student data, encompassing academic performance metrics, behavioral patterns, learning preferences, and real-time

feedback. By combining supervised and unsupervised learning algorithms, the platform constructs a holistic learner profile for each student. This profile is dynamically updated to reflect changes in performance and engagement, serving as the foundation for personalized recommendations regarding curricula, study plans, and learning resources. The system's real-time adaptability ensures that every learner receives a tailored learning approach and enrichment opportunities suited to their current and future needs.

The AI-driven personalized learning path system features an intelligent content recommendation engine that utilizes NLP and deep learning technologies to select the most appropriate materials from a vast repository of educational resources. It identifies and recommends resources—such as videos, interactive simulations, and text-based content—that align with a student's proficiency level, learning preferences, and areas requiring additional support. This not only enhances the relevance of resources but also allows educators to shift their focus from manually curating materials to fostering deeper learning experiences and cultivating student autonomy.

Assessment and feedback components are seamlessly integrated into the platform. AI analyzes quiz responses, assignments, and written reflections to pinpoint a student's level of concept mastery and identify gaps in understanding. Furthermore, the feedback mechanism is enhanced by AI chatbots capable of answering questions, providing explanations, and motivating students through personalized messaging. Progress dashboards present learning trajectories, mastery levels, and areas for improvement through intuitive visualizations, benefiting students, teachers, and parents alike. Scalability and integration capabilities are key strengths of this project. The platform can operate either as a standalone solution or as an enhancement to existing Learning Management Systems (LMS), catering to the needs of both independent learners and classroom environments. Its modular architecture supports continuous expansion into new subjects, grade levels, and educational domains—spanning everything from academic studies to vocational training.

[P05] ShotSense AI: Can AI Coach Your Basketball Shot?

Requirements

This project requires a team with good programming skills and an interest in computer vision, large language models (LLMs), and software engineering. Experience with Python, computer vision, web/mobile development, or AI would be useful, but students are not expected to train large models from scratch.

Background

Basketball players can easily record their practice sessions using a smartphone, but obtaining useful feedback from these videos normally requires manual review or a coach. Recent advances in computer vision can extract information about a player's movement and shooting performance, while LLMs can help transform such information into understandable and personalised feedback.

The challenge is to combine these technologies into a reliable and usable basketball training system.

Outline

This project aims to design and develop an intelligent basketball shooting tutor. The system should analyse recorded basketball shooting videos and provide visual and quantitative information about shooting performance. An existing LLM should then be used to interpret the analysis and generate understandable, personalised feedback for the player.

Possible outputs may include shot counting, body-joint measurements, shot timing, movement consistency, annotated video, make/miss information, and natural-language feedback.

The exact system architecture, computer-vision methods, LLM, measurements, and user interface should be determined by the team through requirements analysis, feasibility studies, prototyping, and evaluation.

Tasks & Deliverables

The selected students will form a software engineering team and carry out the main stages of the software-development process. This will include requirements analysis, investigation of relevant computer-vision and LLM technologies, system design, implementation, integration, testing, and evaluation.

The team will be expected to produce appropriate software-engineering artefacts, source code, documentation, testing and evaluation results, and a working system demonstration.

Additional Information

The project should be achievable using ordinary smartphones, open-source software, pre-trained computer-vision models, and existing LLMs. Students are not required to train large AI models from scratch. Real-time analysis may be considered as an extension if time permits.

[P06] AI Assisted Website Accessibility Testing and Repair

Requirements

Students should have experience with web development, HTML and CSS, databases, Git, and software testing. Training a large model from scratch is not required.

Background

A website can look usable while presenting barriers to people using screen readers, keyboard navigation, or enlarged text. Examples include unlabelled form fields, images without suitable text alternatives, and poor text contrast. Existing test engines can detect some common defects, but manual assessment remains necessary [1, 2].

This project combines automated checks with AI explanations and proposed code changes. Users see the evidence for each finding, review the proposed repair, and test the revised page before exporting it.

Outline

Users can upload a small website made with HTML, CSS, and images. The software checks the pages for accessibility problems and lists them by page and problem type. It explains each problem and suggests a fix. Users can accept, change, or reject each suggestion. The software applies approved changes to a copy of the website. Users can preview the changes before the software checks the website again and provides the updated files and a report.

For example, a student society uploads its event registration website. The software finds an email input box with no label and a poster with no text description for screen readers. AI suggests a label for the email box and a description of the poster based on the image and nearby text. The society reviews the wording, previews the changes, and checks that the form still works correctly.

Use a modular design so the browser scanner, AI service, repair editor, and report generator can each be tested on their own. AI should suggest changes. The software should check that each suggestion is in the correct format and apply only changes it supports. Users must be able to use the dashboard with a keyboard, and all buttons and other controls must have clear labels.

Tasks & Deliverables

The team should follow a complete software engineering process, starting with stakeholder interviews and agreed acceptance criteria. The software should include the following functions.

- Website import and project management
- Automated accessibility inspection
- AI explanations and repair proposals
- Human review and preview
- Repair verification
- Reporting and export

Final deliverables include,

- A software with sample websites and clear instructions for installing and running it.
- Source code and a record showing which tests check each requirement.
- Description of test examples, evaluation results, and a list of known limitations.
- A user guide, a demonstration video, and the report required by the module.

Additional Information

There are no restrictions on the operating system or programming languages used in this project. The team may choose suitable technologies based on the project requirements and their skills. They should explain their choices and provide clear instructions for installing and running the software.

References

[1] Playwright. Accessibility testing. https://playwright.dev/docs/accessibility-testing

[2] W3C Web Accessibility Initiative. Evaluating Web Accessibility Overview. https://www.w3.org/WAI/test-evaluate/

[P07] A Software Tool for Qualitative Spatial and Temporal Reasoning

Requirements

This project will require good knowledge of discrete mathematics and good programming skills.

Background

In our daily life, almost all the information systems involve some spatial or temporal information. For example, a geographic information system often provides locations of real-world objects using a map. Representing spatial and temporal information qualitatively is more intuitive for human beings. For instance, it is more natural to state that a university is to the south of a railway station, rather than providing the precise angle formed by the objects. There exist many qualitative spatial or temporal calculi which formalise qualitative spatial or temporal relations, as well as their reasoning rules.

Outline

This project aims to develop a software tool for qualitative spatial and temporal reasoning. The software tool should implement several main algorithms to support the reasoning with topological, direction and distance relations, as well as temporal relations.

Tasks & Deliverables

- Provide an expanded description of the problem to be solved, identify the main challenges in qualitative spatial and temporal reasoning.
- Review existing similar software tools and analyse their advantages and limitations, write this up in a chapter of literature review.
- Specify functional and non-functional requirements, write this up in a chapter of requirement specification.
- Design a software making use of the knowledge and skills learnt in software engineering modules, write this up in a chapter of software design.
- Implement the software using Java, C/C++, Python or other programming languages and write this up in a chapter of software implementation.
- Evaluate and test the implemented software making use of the knowledge and skills learnt in software engineering modules, write this up in a chapter of evaluation and testing.

- Provide a user manual of the software.
- Work in a team, collaborate and learn from each other. Write an individual reflection report.

For more details, please refer to the GRP module handbook.

[P08] Text2Visual: An Intelligent System for Transforming Textual Content into Visual Narratives

Requirements

This project requires a team interested in building an end-to-end software system that combines text understanding, multimodal content generation and interactive user interfaces. Students should be comfortable with web or mobile development, working with APIs, databases and basic software engineering practices. Experience with natural language processing or computer vision would be useful, but is not essential. The team should be willing to explore existing language and image/video generation models rather than attempting to train large models from scratch.

Background

A great deal of useful information is still presented primarily as text: textbook explanations, stories, biographies, public-information materials, reports and other long-form content. Although text can be precise, it is not always the most engaging or accessible way to communicate ideas. Visual storytelling can make sequences, relationships and key concepts easier to follow, particularly when the content contains several events, characters or stages.

Recent multimodal and generative models make it possible to analyse text, identify its important elements and create coherent visual content from it. However, producing a useful visual narrative involves much more than simply sending a paragraph to an image generator. The system must decide what is important, break the content into suitable scenes, preserve continuity, generate appropriate captions or narration, and allow the user to review and correct the result. This project explores how these components can be brought together into a practical and enjoyable software application.

Outline

The aim of this project is to design and develop an intelligent system that transforms textual content into an engaging visual narrative. The team may choose an application domain that interests them, such as education, literature, science communication, biography, history, public awareness or another suitable area.

A user should be able to provide a passage of text or a short document. The system will analyse the material, identify key entities, concepts, events and relationships, and organise them into a sequence of scenes or visual units. It will then generate a visual representation of the content. Depending on the team's chosen design, the final output might be a comic, illustrated storyboard, animated sequence,

narrated visual story, short video, or another appropriate multimedia format.

The system should not behave as a one-click black box. Users should be able to inspect the generated structure, edit scene descriptions, change captions or narration, regenerate individual scenes, rearrange the sequence and refine the final output. Stronger teams may also explore features such as consistent characters or visual style, source-to-scene traceability, multilingual narration, automatic voice-over, or different visualisation styles for different types of content.

Tasks & Deliverables

The selected students will work as a software engineering team and are expected to complete a full development cycle. The exact implementation is intentionally open so that the team can make its own design and technology decisions. Expected activities and deliverables include:

- Investigate potential users and application scenarios, and identify where converting text into visual narratives would be useful.
- Review existing text-to-image, text-to-video, storyboard and multimodal generation tools, and define a realistic project scope.
- Elicit and document functional and non-functional requirements for the proposed system.
- Design the overall architecture, including text analysis, scene/storyboard generation, visual generation, data storage and the user interface.
- Implement a pipeline that extracts the important information from input text and converts it into a structured sequence of visual scenes.
- Integrate suitable existing models or APIs to generate visual content. The project does not require training a large generative model from scratch.
- Develop an interactive editor so that users can review, modify, reorder or regenerate the generated scenes rather than accepting the output blindly.
- Support the creation and export of a complete visual narrative in the format selected by the team.
- Test the system thoroughly, including functional testing, usability testing and evaluation of how well the generated visual output represents the original text.
- Produce appropriate software engineering documentation, source code, test evidence, user guidance and a working demonstration of the final system.

Additional Information

The project is deliberately not restricted to one type of source material or one visual format. Teams are encouraged to choose a focused use case that they can implement well and demonstrate convincingly. For example, one team might turn difficult textbook passages into illustrated explanations, while another might transform a short narrative into an animated visual story.

There are many existing language, image and video generation models, open-source libraries and APIs that can be used as building blocks, so the main challenge is expected to be the design of a coherent, usable and well-engineered system. Particular attention should be paid to preserving the important meaning of the source text, maintaining consistency across scenes, giving users control over generated content, and evaluating whether the resulting visual narrative actually improves engagement or understanding.

Possible extensions include multilingual input/output, speech narration, automatic subtitles, different visual styles, collaborative editing, source traceability, accessibility features, and user studies comparing text-only content with generated visual narratives.

[P09] Creating a Sustainable Workflow for Virtual Production (VP) Filmmaking

Requirements

The University has been approached to support the realisation of a short film using a studio for ARRI Virtual Production (VP) and Motion Capture (MC), with an explicit request to consider sustainability during the production process.



Figure 1: ARRI VP and MC

The Software Engineering (SE) team must be motivated, attentive to detail, capable of working both independently and collaboratively, and eager to explore and innovate to support the creative realisation of the film, while also considering sustainability. The team will engage with VP technologies, including elements not yet covered in the University's Computer Science programmes.

The project vision includes:

- Engaging from the pre-production stage to devise a bespoke workflow that ensures the successful realisation of these short films.
- Creating a series of highly realistic and innovative virtual assets using Unreal Engine, serving as the cornerstone of the production, projected on an LED Wall (see *figure 1* as example).

- Developing a plug-in to synchronise real and virtual lighting in a VP and MC Studio to reduce energy consumption during the filming process.
- This resource will inspire the film crew to produce the short films and tackle the challenges of this ground-breaking filmmaking technology.
- Additional opportunities exist for the SE team to support key creative stakeholders during the production process in December 2026/January

  2027.

Finally, this project will investigate how Virtual Production (VP) is reshaping storytelling practices by future-proofing screen production workflows, developing skills in revolutionary technologies, and enhancing sustainability practice.

Background

The clients and stakeholders bring a wealth of experience in producing traditional short films that involve guiding novice filmmakers to create professional content that resonates with international audiences. Their expertise now extends into VP, aiming to produce ground-breaking short films using this revolutionary technology, to support its maturity and adoption across the global film industry.

Outline

The selected students will form an SE team and work together to complete a full cycle (maybe several full cycles) of the SE process. Relevant stakeholders, their associates, and industry partners, will be identified, and *requirements engineering* processes used to identify the exact SE project requirements. A final project proposal will be prepared, and the project implemented over the course of the GRP time period.

Because the SE team will need to go through *requirements engineering*, part of this project will involve applying an appropriate requirements elicitation methodology. The requirements engineering, feasibility analysis, and project scoping will all need to be completed early in the GRP time frame.

The team will need to make informed decisions about which SE approaches or methodologies to apply to this project. (Experiences from previous SE teams may prove useful.)

Over the course of the project, the SE team will need to produce several deliverables, which may include (but are not limited to):

- a report on the current situation
- a proposed system (or systems) to implement

- a system requirements specification
- design documentation
- team management and planning documentation
- prototypes
- progress reports
- verification and validation plans
- code and code documentation
- tests and test reports
- instructions/user manual(s)
- an Open Education Resource (OER)

The target goal of the SE project will be to deliver, on time, a system (or several systems) that meet the requirements of the various stakeholders identified over the course of the project.

Tasks & Deliverables

This unique project may have several groups of stakeholders, and will require significant project management skills to complete. In addition to the deliverables listed above, the team may also need to produce other resources, including recommended protocols or format specifications, or other advice for future evolution. A workstation with Unreal Engine will be available to the team.

References

[R-1] https://en.wikipedia.org/wiki/Open_educational_resources

[P10] ErgoAssist: A Specialised LLM for Anthropometric Seating Design

Requirements

This project will require a team with an interest in artificial intelligence, natural language processing, and human-centred design. The team should be comfortable with Python, APIs, and modern LLM tooling (e.g., LangChain, vector databases). Familiarity with prompt engineering or retrieval-augmented generation (RAG) is beneficial but not essential—the team will learn these during the project. Domain knowledge of ergonomics or anthropometry is helpful, but the team will acquire this through stakeholder engagement and independent study.

The project involves regular interaction with stakeholders (designers, ergonomists, and Mr X) to elicit requirements, validate design decisions, and evaluate the system. Strong communication and project management skills will be essential.

Background

Designers are increasingly expected to balance creativity with evidence-based practice. Yet integrating formal ergonomic knowledge—particularly anthropometric data—into routine design work remains inconsistent. Many designers lack structured training in human factors, and those who have it often struggle to apply it efficiently under real-world constraints.

Large language models (LLMs) offer a promising solution. Unlike traditional CAD tools, LLMs can reason in natural language and engage in dialogue that mirrors how designers actually work. However, general-purpose LLMs lack the specialised knowledge needed to give reliable ergonomic guidance. Retrieval-augmented generation (RAG) can ground LLM outputs in authoritative sources, making domain-specific assistance feasible.

Seating design is an ideal proof-of-concept domain: it is data-rich (anthropometric standards and handbooks are widely available), well-bounded (seat height, depth, backrest angle, lumbar support), and visually intuitive. This makes it accessible to Computer Science students while still presenting genuine technical and domain challenges.

Outline

This project aims to develop **ErgoAssist**, a proof-of-concept LLM assistant that helps designers apply anthropometric principles in seating design. The system will interpret a designer's natural-language description of a seating problem, retrieve relevant anthropometric data, and generate ergonomically informed recommendations grounded in authoritative sources.

The primary technical approach will be **retrieval-augmented generation (RAG)**, with a curated vector database of anthropometric standards, handbooks, and research papers. Lightweight fine-tuning (e.g., LoRA) may be explored as a stretch goal if time permits.

Key research questions include:

- How can LLMs be adapted to provide reliable, traceable anthropometric guidance for seating design?
- How can designers interact with such a system in a way that supports—rather than replaces—their professional judgment?
- How can the system's outputs be evaluated for ergonomic soundness?

Expected tasks and deliverables include:

- A working prototype with a conversational interface.
- A curated knowledge base of anthropometric sources.
- Evaluation of the system against published standards and, where possible, expert review.
- Documentation of the system's capabilities, limitations, and ethical considerations.

Tasks & Deliverables

The selected students will form a software engineering (SE) team and work together to complete a full cycle of the SE process. Relevant stakeholders (Mr X, designers, and potentially ergonomists) will be identified, and requirements engineering processes used to identify the exact SE project requirements. A final project proposal will be prepared, and the project implemented over the course of the GRP time period.

Because the SE team will need to go through requirements engineering, part of this project will involve applying an appropriate requirements elicitation methodology. The requirements engineering, feasibility analysis, and project scoping will all need to be completed early in the GRP time frame.

The team will need to make informed decisions about which SE processes, approaches or methodologies to apply to this project. Experiences from previous SE teams may prove useful.

Over the course of the project, the SE team will need to produce several deliverables, which may include (but are not limited to):

- a report on the current situation
- a proposed system to implement
- a system requirements specification

- design documentation
- team management and planning documentation
- prototypes
- progress reports
- verification and validation plans
- code and code documentation
- tests and test reports
- instructions/user manual(s)

The target goal of the SE project will be to deliver, on time, a system that meets the requirements of the various stakeholders.

Additional Information

The project is deliberately scoped as a proof-of-concept. The team is not expected to build a production-ready system or to cover all of ergonomics—depth over breadth is the priority. Anthropometric data sources (e.g., ISO standards, *Bodyspace*) will be provided or identified during requirements engineering.

This project has strong potential for academic dissemination in AI, HCI, and design research. A small-scale user study with designers would strengthen the work considerably.

Valuable tips for initiating this project include:

- Identifying which seating design variables (seat height, depth, backrest angle, etc.) are most valuable to designers.
- Curating a focused, high-quality knowledge base rather than attempting to cover all anthropometric data.
- Building evaluation into the project from the start—technical, expert, and user evaluation.
- Documenting limitations clearly; the system advises, the designer decides.

[P11] Level Up: Serious Games for 21st-Century Employability Skills

Requirements

This project will require a team with an interest in game design, educational technology, and human-centred design. The team should be comfortable with game engines (e.g., Unity3D, Unreal Engine), 3D tools (e.g., Blender), and/or modern AI tooling (e.g., LLM APIs, adaptive learning systems). Familiarity with instructional design or pedagogical theory is beneficial but not essential—the team will learn these during the project.

The project involves regular interaction with stakeholders (educators, students, and Mr X) to elicit requirements, validate design decisions, and evaluate the games. Strong communication and project management skills will be essential.

Background

The landscape of higher education is evolving rapidly. Employers consistently report that graduates possess strong technical knowledge but often lack the soft skills—communication, collaboration, critical thinking, and creativity—that are essential for success in modern workplaces. This skills gap has prompted educators to explore innovative pedagogical strategies that foster both technical proficiency and employability skills.

Serious games have emerged as a promising avenue for addressing this gap. By combining engaging gameplay with intentional learning objectives, serious games can create immersive experiences that promote skill development in contexts that are safe, scalable, and engaging. Unlike conventional instructional methods, serious games leverage intrinsic motivation, immediate feedback, and narrative-driven challenges to sustain learner engagement and deepen understanding.

Recent advances in artificial intelligence (AI), including large language models (LLMs) and adaptive learning algorithms, have opened new frontiers for serious games. These technologies can enable personalised learning experiences, dynamic difficulty adjustment, real-time feedback, and intelligent NPC behaviour—all of which can enhance engagement and learning outcomes.

Outline

This project aims to develop a **serious game** for higher education students that targets a **focused subset** of 21st-century employability skills. Rather than attempting to address all skills within a single game, the team will select a small number of skills (e.g., critical thinking, collaboration, communication) and design game mechanics, narratives, and interactions explicitly mapped to those outcomes.

The scope is intentionally flexible and may encompass:

- **AI-powered serious games** that leverage LLMs, adaptive learning, or intelligent tutoring systems.
- **Extended reality (XR) experiences** including VR, AR, or mixed reality (MR).
- **Traditional digital games** enhanced with modern learning analytics and feedback mechanisms.
- **Hybrid approaches** that combine multiple technologies and modalities.

Key research questions include:

- How can serious games be designed to develop specific employability skills in higher education students?
- How can game mechanics and narratives be mapped to skill development outcomes?
- How can the effectiveness of the game be evaluated?

Expected tasks and deliverables include:

- A working prototype of a serious game.
- Documentation of the game design, including explicit mapping of game elements to skill outcomes.
- Evaluation of the game's effectiveness through user testing.
- Documentation of the game's capabilities, limitations, and ethical considerations.

Tasks & Deliverables

The selected students will form a software engineering (SE) team and work together to complete a full cycle of the SE process. Relevant stakeholders (Mr X, educators, and students) will be identified, and requirements engineering processes used to identify the exact SE project requirements. A final project proposal will be prepared, and the project implemented over the course of the GRP time period.

Because the SE team will need to go through requirements engineering, part of this project will involve applying an appropriate requirements elicitation methodology. The requirements engineering, feasibility analysis, and project scoping will all need to be completed early in the GRP time frame.

The team will need to make informed decisions about which SE processes, approaches or methodologies to apply to this project. Experiences from previous SE teams may prove useful.

Over the course of the project, the SE team will need to produce several deliverables, which may include (but are not limited to):

- a report on the current situation
- a proposed system to implement
- a system requirements specification
- design documentation
- team management and planning documentation
- prototypes
- progress reports
- verification and validation plans
- code and code documentation
- tests and test reports
- instructions/user manual(s)

The target goal of the SE project will be to deliver, on time, a system that meets the requirements of the various stakeholders.

Additional Information

The project is deliberately scoped to focus on a **focused subset** of 21st-century employability skills—depth over breadth is the priority. The team will select which skills to target during requirements engineering, based on stakeholder input and feasibility.

This project has strong potential for academic dissemination in educational technology, game studies, and HCI. A small-scale user study with students would strengthen the work considerably.

Valuable tips for initiating this project include:

- Identifying which employability skills are most valuable to higher education students and educators.
- Selecting a game genre and technology stack that align with the targeted skills and available resources.
- Building evaluation into the project from the start—technical, educational, and user evaluation.
- Documenting limitations clearly; the game is a learning tool, not a replacement for formal education.

[P12] CampusWorld: A Smart Campus World Model Sandbox

Requirements

Students should have a solid foundation in computer science and information technology, including programming in languages such as Python or JavaScript, basic web development, and database design. They should understand simple software architecture concepts such as APIs, client-server systems, and event-driven communication, because these are needed to connect different campus data sources and user interfaces. Familiarity with cloud platforms or deployment tools is helpful, but students can also run the software locally.

Students should also be comfortable working with data, including collecting, cleaning, and storing simple datasets. Basic knowledge of machine learning or simulation is useful, but advanced AI experience is not required. The project is designed for undergraduate teams, so simple rule-based models, small prediction models, and clear explanations are acceptable. Teamwork, communication, and a willingness to learn new tools are more important than having every technical skill at the beginning.

Background

A smart campus uses data and technology to improve how universities manage resources, spaces, energy, and daily services. Traditional campus dashboards can show what is happening now, but they often cannot answer "what if" questions. A world model goes one step further by learning or simulating how a campus changes over time, predicting future conditions, and testing different decisions in a virtual environment. For example, it can estimate how a change in class dismissal time may affect cafeteria queues, or how classroom usage may influence energy consumption. CampusWorld is a small-scale world-model sandbox inspired by digital twin ideas. The goal is to build a software that lets users adjust parameters, see predicted future outcomes, and receive practical suggestions.

Outline

The project is organized into four phases. The first phase, Discovery and Planning, focuses on choosing a campus scenario, collecting or generating sample data, and defining clear user stories and acceptance criteria. The second phase, Architecture and Design, involves designing the system architecture, database, APIs, and user interface. The third phase, Development and Integration, includes building the simulation, prediction, counterfactual analysis, and visualization components. The fourth phase, Pilot and Testing, involves testing the software with realistic scenarios, gathering feedback, and improving the system.

Tasks & Deliverables

The team must design and implement a software called CampusWorld. The system should allow users to model a selected campus area, advance time, predict future conditions, and compare "what if" scenarios against a baseline. It should also provide simple decision suggestions, such as opening an extra cafeteria window, changing a class schedule, or adjusting classroom energy use. A visual dashboard should present maps, heatmaps, trend charts, or comparison views so that users can understand the results easily.

Additional Information

Students should focus on good software design, clear assumptions, and a working demonstration rather than perfect prediction accuracy. Simple rule-based models or small machine-learning models are acceptable when they are well explained and properly tested.

[P13] AI-Powered Personalized Learning System for Secondary School Students

Requirements

Develop a responsive web application that helps junior and senior secondary school students improve academic performance through AI-assisted diagnosis, personalized learning, and continuous assessment. The system should support everyday learning and examination preparation, without being limited to the Gaokao.

Core features include:

- Uploading and privately storing learning materials in image and PDF formats.
- Recognizing questions and student responses, with manual correction.
- Identifying knowledge gaps and explaining mistakes using supporting evidence.
- Generating personalized study plans, explanations, and differentiated exercises.
- Assessing progress through independent practice, transfer tasks, and delayed retesting.
- Providing learning records, AI tutoring, access control, and data deletion.

The implementation must use working AI services and persistent storage, with explicit handling of processing failures and uncertain results.

Background

Students often receive grades and correct answers without understanding why they made mistakes or how to improve. Teachers may also lack the time to provide individualized feedback consistently.

This project connects learning evidence with targeted support and repeated assessment. An undergraduate software engineering team should initially focus on one subject and a limited range of grades, while designing for future expansion.

Outline

The system follows a diagnosis–learning–practice–assessment workflow:

1. Students upload learning materials and confirm the recognized content.
2. The system analyzes their responses and identifies potential knowledge gaps.
3. Students receive targeted explanations, study tasks, and graded exercises.
4. Independent assessments and delayed retesting check learning outcomes.

5. Results update the student's learning profile and inform the next study cycle.

The architecture should separate user interaction, document processing, AI services, learning management, and data storage. Processing status and learning progress should persist across interrupted sessions.

Tasks & Deliverables

The team should complete the following software engineering activities:

1. Requirements Analysis: Define user scenarios, functional and non-functional requirements, project scope, and acceptance criteria.
2. System Design: Produce architecture diagrams, database schemas, interface specifications, and workflow models.
3. Implementation: Develop material upload, content recognition, learning diagnosis, personalized practice, progress tracking, and AI tutoring.
4. Content Management: Organize authorized questions, explanations, and assessment criteria by grade, subject, and knowledge point.
5. Testing: Conduct unit, integration, and end-to-end tests, including AI uncertainty, upload failures, access control, and data recovery.
6. Delivery: Submit executable source code, database initialization scripts, deployment instructions, a test report, a README, and a handover document.

Additional Information

The design should address several practical constraints:

- Missing or incomplete solution steps should trigger clarification or supplementary exercises rather than unsupported diagnoses.
- Knowledge mastery should be assessed using repeated evidence, not a single correct answer.
- AI-generated learning content should be clearly identified and validated before inclusion in the shared content library.
- Student data should be minimized, protected, and allow users to delete it.
- Additional subjects, grade levels, and teacher-facing features are optional extensions, not initial delivery requirements.

[P14] Software Tool for Fairness in Machine Learning

Ensuring that decision support systems based on machine learning are fair, avoiding discrimination and bias is an important component for building responsible AI systems. In many application domains such as healthcare and financial services, fairness may be a legal requirement, and organizations using machine learning need to demonstrate fairness in automated decision making. Fairness is well-understood in computer science and is measurable. However, existing tools to measure fairness are somewhat difficult to use and do not provide a good interface for an end-user who may not be technically minded. Additionally, although fairness can be measured, it is less clear how to discover patterns of bias in data. To address these deficiencies, in this project you will develop software that supports fairness measurement and bias discovery. The deliverables will be in two parts:

(1) Develop an underlying Python package to implement fairness measures and bias detectors in datasets.

(2) Develop a GUI that allows a non-technical user to upload a dataset and receive a fairness report and dashboard.

Background reading:

Fairness and machine learning: Limitations and Opportunities by Solon Barocas, Moritz Hardt and Arvind Narayanan

https://fairmlbook.org/

[P15] Interactive Multi-UAV Viewpoint and Path Visualization Tool

Requirements

The project aims to develop a multi-UAV viewpoint and path visualization tool with the following key requirements:

- Multi-UAV Path Analysis: Enable users (e.g., decision-makers) to visualize and assess pre-generated flight paths and viewpoints for multiple UAVs.
- Manual Adjustment: Allow users to interactively modify flight paths and viewpoints for individual UAVs, sub-groups, or the entire fleet as needed, based on visual feedback. For example, move the viewpoint assignment from UAV1 to UAV2.
- Dynamic Multi-UAV Visualization: Provide a dynamic simulation of multiple UAVs flying through the planned viewpoints.
- Export Functionality: Support exporting refined per-UAV paths and coordinated fleet schedules in formats compatible with UAV platforms such as DJI for real-world execution.

Background

3D reconstruction of large-scale and complex objects with high levels of detail using data captured by UAVs is gaining significant attention, particularly in industrial applications such as urban modeling, cultural heritage documentation, and construction site inspection. Multiple UAVs offer unique advantages, including autonomous data acquisition, the ability to fly at varying altitudes, and access to a wide range of vantage points, making them highly suitable for capturing detailed imagery of large structures.

However, to fully leverage these capabilities, effective multi-UAV viewpoint and flight path planning is essential. Recent advancements have led to the development of algorithms that generate optimal viewpoints and flight trajectories for multiple UAVs. While these approaches show promise, current visualization tools for supporting such plans are often static and limited in interactivity.

Outline

In practical scenarios, a dynamic visualization tool would be far more beneficial. Such a tool could simulate multiple UAVs' flight paths as they move through planned viewpoints and capture images. This would not only help in evaluating the effectiveness of the paths but also enhance understanding of the data acquisition process.

Moreover, existing static outputs lack user interaction capabilities. It is important to allow end-users to manually adjust viewpoints and flight paths for individual UAVs or the whole fleet and immediately visualize how these changes affect the 3D reconstruction quality, specifically in terms of accuracy and completeness, as well as the efficiency of the paths in terms of per-UAV and fleet energy consumption, mission duration, and coordination.

User input is often more effective in practice than fully automated plans, especially when guided by domain knowledge or specific project requirements. Therefore, providing users with the ability to refine multi-UAV flight plans before execution is essential for improving both the reconstruction results and operational efficiency.

Tasks & Deliverables

Tasks

Requirements Analysis

- Gather and define functional and non-functional requirements for multi-UAV visualization, coordination, and export.
- Identify supported UAV platforms (e.g., DJI SDK compatibility) and constraints for simultaneous or coordinated multi-UAV missions.

Visualization Tool Development

- Develop a 3D visualization system for multiple UAV flight paths and camera frustum representation at waypoints, clearly illustrating coverage and geometric constraints of image capture for each UAV and the fleet.
- Allow user-defined UAV/camera models and parameters (e.g., focal length, FOV, sensor size) for individual UAVs or fleet templates to dynamically generate frustums and simulate mission-specific imaging behavior.
- Support loading and rendering of 3D environmental models (e.g., terrain, buildings) to provide spatial context and facilitate collision-aware path planning and deconfliction among multiple UAVs.
- Simulate multiple UAVs flying along planned paths with dynamic visualization of camera frustum orientation and image capture points, including interactive playback, per-UAV/fleet viewpoint control, and synchronized time-based animation.

Manual Multi-UAV Path Adjustment Interface

- Develop interactive tools for users to modify flight paths and viewpoints for individual UAVs, sub-groups, or the entire fleet.
- Include quantitative feedback on reconstruction quality and flight efficiency, including per-UAV and fleet-level energy consumption, mission makespan, coverage, overlap, redundancy, collision risk, and communication connectivity.

Integration with UAV Platforms

- Implement import functionality to load and visualize existing multi-UAV flight paths or paths exported by this tool.
- Implement export functionality to generate standardized flight plans for each UAV (e.g., KML, CSV, GPX, or mission-specific formats), plus a coordination manifest/schedule compatible with UAV platforms (e.g., DJI).

Testing and Validation

- Test the tool with sample multi-UAV flight plans.
- Validate usability, accuracy, coordination, and integration with target platforms.

Documentation

- Provide user guide and technical documentation for multi-UAV path refinement and export.
- Include examples and instructions for individual and fleet-level path refinement and export.

Deliverables

Interactive Multi-UAV Visualization Tool

- Standalone or web-based application for multi-UAV path and viewpoint visualization.

Manual Multi-UAV Adjustment Interface

- User interface for modifying and optimizing paths and viewpoints for individual UAVs and fleets.

Flight Plan Export Module

- Feature to export refined paths for multiple UAVs in a DJI-compatible format (e.g., KML, GPX, or via DJI SDK), including per-UAV mission files and fleet coordination data.

Documentation

- User manual
- Technical documentation

- Setup and deployment instructions

Test Data and Use Cases

- Sample multi-UAV flight plans and reconstruction scenarios for demonstration and validation.

Additional Information

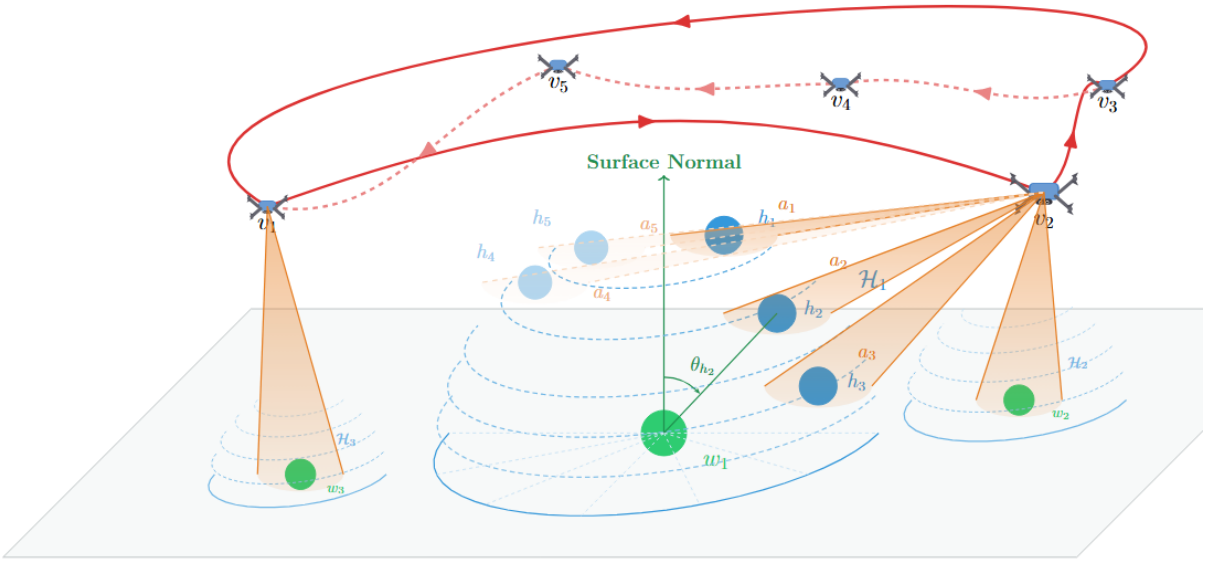


Figure 2: UAV flight path

[P16] PhishGuard: An Explainable Machine-Learning Phishing Detection and Security Awareness Platform

Requirements

This project will require a team with an interest in machine learning, cybersecurity, and web application development. In other words, this project is to build a software tool that uses machine learning to detect phishing links or emails, explain why they look suspicious, and help users learn how to avoid online scams. So, it is part detector, part explainer, and part security-awareness trainer. The selected team should be willing to develop technical proficiency in:

- Python programming and data processing
- Machine learning libraries such as scikit-learn, pandas, NumPy, and optionally XGBoost or PyTorch
- Web backend and Frontend development
- Database design and basic authentication/security practices
- Browser extension development as an optional stretch goal
- Explainable AI tools such as SHAP or LIME

The project must be conducted ethically. The team will use public or synthetic phishing datasets only. They must not conduct live phishing attacks, collect real credentials, visit malicious URLs directly, or perform unauthorised network scanning. Any user testing must follow University research ethics and data protection requirements.

Background

Phishing remains one of the most common and damaging cybersecurity threats. Attackers use fraudulent emails, messages, and websites to trick users into revealing passwords, financial details, or other sensitive information. Traditional defences often rely on blacklists and rule-based filters, but these can struggle against new or rapidly changing phishing campaigns.

Machine learning offers a promising alternative because it can learn patterns from large collections of known phishing and legitimate examples. However, many ML-based detectors operate as “black boxes”: they produce a warning but do not explain why a URL or email is considered suspicious. This can reduce user trust and make it harder for people to learn how to recognise phishing attempts.

This project aims to build a software platform that combines machine-learning phishing detection with clear explanations and security-awareness features. The system will classify suspicious URLs

and emails, show the most important features behind each prediction, and provide an interface that helps users understand and report potential threats.

Outline

The primary aim of this project is to design, implement, and evaluate an explainable machine-learning phishing detection and awareness platform. The system should be usable by ordinary users as well as by an administrator or security-awareness coordinator.

Key research questions include:

- Which machine-learning models best balance detection accuracy and false-positive rate on public phishing datasets?
- How can explanations such as SHAP or LIME be presented to non-expert users in a way that improves trust and understanding?
- What user-interface and awareness features encourage safer behaviour when a suspicious link or email is detected?
- How can a browser-extension or web-based prototype be designed to protect user privacy while still providing useful warnings?

The project should produce a working software prototype, not only a model. Expected components include:

- A data pipeline for cleaning and preparing public phishing and legitimate URL/email datasets
- A machine-learning module that trains and compares several classifiers
- An explainability module that identifies the features influencing each prediction
- A backend API that accepts a URL or email text and returns a classification, confidence score, and explanation
- A user-facing web dashboard for scanning, history, statistics, and awareness guidance
- An optional browser extension that warns users about suspicious links
- A testing and evaluation package covering model performance, usability, and ethical considerations

Tasks & Deliverables

The selected students will form a software engineering team and complete a full software engineering cycle, or several cycles, over the GRP period. They will identify stakeholders, apply requirements

elicitation methods, perform feasibility analysis, and produce a final project proposal before implementation.

The team will need to make informed decisions about which software engineering processes, approaches, or methodologies to apply. Deliverables may include, but are not limited to:

- A report on the current situation and related work
- A system requirements specification
- Design documentation, including architecture and user-interface design
- Team management and planning documentation
- Prototypes and progress reports
- Verification and validation plans
- Source code and code documentation
- Tests and test reports
- Instructions and user manual(s)
- An Open Educational Resource (OER) that explains the project's approach to ML-based phishing detection

Technical tasks and deliverables should include:

1. Dataset preparation
    - Collect public phishing and benign datasets, such as PhishTank, OpenPhish, Kaggle phishing email datasets, and legitimate email corpora.
    - Clean, balance, and split the data into training, validation, and test sets.
    - Document ethical and legal constraints on dataset use.
2. Feature engineering
    - Extract URL-based features such as length, number of dots, hyphens, HTTPS usage, IP address usage, suspicious top-level domains, and URL shortener presence.
    - Extract email-based features such as keyword frequency, HTML content, link count, and sender-domain characteristics.
    - Optionally use TF-IDF or embeddings for email text.
3. Machine-learning modelling

- Train and compare at least three models, for example Logistic Regression, Random Forest, XGBoost, SVM, or a simple neural network.
- Evaluate using accuracy, precision, recall, F1-score, ROC-AUC, and false-positive rate.
- Investigate class imbalance and robustness against unseen phishing patterns.

4. Explainability
    - Integrate SHAP, LIME, or a comparable method.
    - Display the top contributing features for each prediction in a user-friendly format.
    - Evaluate whether explanations improve user trust and comprehension.
5. Backend API
    - Implement endpoints such as /predict/url and /predict/email.
    - Return label, confidence, explanation, and timestamp.
    - Apply secure coding practices, input validation, and privacy-preserving logging.
6. Frontend and user experience
    - Build a web dashboard for scanning URLs/emails, viewing history, and understanding results.
    - Include awareness content such as phishing warning signs, quizzes, and reporting guidance.
    - Optionally implement a browser extension that warns users about suspicious links.
7. Testing and evaluation
    - Conduct unit, integration, system, and usability testing.
    - Run a small user study with students or staff to assess trust, comprehension, and false-positive tolerance.
    - Document results and limitations.
8. Documentation and OER
    - Provide installation instructions, API documentation, and a user manual.
    - Create an OER that explains how ML can be used for phishing detection and awareness.

Additional Information

This project is suitable for a Year 3 undergraduate team because it uses public datasets, standard ML libraries, and a bounded web/browser prototype rather than live offensive security operations. The focus is on software engineering, machine learning, human-computer interaction, and usable security.

Important ethical and legal considerations:

- Use only public or synthetic datasets.
- Do not collect real credentials, personal data, or live phishing content.
- Do not visit or interact with live malicious URLs.
- Follow University research ethics, informed consent, and data protection requirements.
- Apply secure development practices and privacy-preserving design.

Potential tools and technologies:

- Python, pandas, NumPy, scikit-learn, XGBoost, SHAP, LIME
- FastAPI or Flask, SQLite or PostgreSQL, React or Vue
- Git, Docker, GitHub Actions for testing
- Browser extension APIs for optional link warnings

Potential academic dissemination:

- Usable security and privacy
- Explainable AI for cybersecurity
- Human-computer interaction and security awareness
- Machine learning for phishing detection

This project has strong potential for a demonstration, a promotional digital artefact, and an OER. It also provides clear opportunities for user testing and empirical evaluation.

References

PhishTank: https://phishtank.org/

OpenPhish: https://openphish.com/

Anti-Phishing Working Group: https://apwg.org/

scikit-learn: https://scikit-learn.org/

SHAP: https://github.com/slundberg/shap

LIME: https://github.com/marcotcr/lime

Open Educational Resources: https://en.wikipedia.org/wiki/Open_educational_resources

[P17] An Intelligent Auto-Clamping Car Mount with Pressure-Sensitive Gesture Recognition

Requirements

This project will require a multidisciplinary team of 5-6 CS students, combining expertise in software engineering, embedded systems design, sensing, and artificial intelligence. The team will develop technical proficiency in microcontrollers or embedded GPU, sensor integration (pressure, capacitive, or infrared proximity sensors), actuator control (servo or stepper motors), and embedded programming. Furthermore, skills in mechanical design (CAD modeling, 3D printing), and basic mobile application development for configuration purposes will be essential. An understanding of automotive safety, driver distraction metrics, and user-centered design is highly beneficial.

Background

The evolution of in-car phone mounting solutions has moved from simple clips to magnetic mounts. While magnetic mounts offer convenience, they pose significant risks to modern smartphones. Strong magnetic fields can interfere with internal compasses (magnetometers), damage NFC modules, disrupt wireless charging coils, and potentially degrade battery health or sensitive electronic components over time. Conversely, traditional mechanical clamping mounts require drivers to use hand to pry the clamps open and insert the phone. This manual interaction creates a major safety hazard, as it divides the driver's attention and requires taking their eyes off the road during a critical moment.

The rapid advancement of low-power micro-sensors and embedded processing capabilities provides an opportunity to bridge this gap. By implementing smart sensing mechanisms, a car mount can become an active, responsive assistant rather than a passive, rigid tool.

Outline

The primary aim of this project is to design, prototype, and evaluate a **intelligent, sensor-driven car phone mount** that automatically secures and releases a smartphone based on the user's physical interaction, without requiring manual manipulation of clamps or exposing the device to magnetic fields.

Key engineering and research questions include:

- How can pressure or gesture sensors accurately distinguish between a user placing a phone and a user retrieving a phone in a dynamic, vibrating vehicular environment?

- What mechanical and electronic designs are required to ensure reliable, silent, and rapid clamping/unclamping using low-power actuators?
- How does the automatic response time of the mount affect driver distraction and overall user experience compared to traditional mounts?

The project entails developing a fully functional software-hardware integrated prototype that enhances driving safety and convenience. Expected tasks and deliverables include:

- Designing a pressure-sensitive or gesture-sensing mechanism (e.g., utilizing force-sensitive resistors, capacitive touch, or ultrasonic sensors) that detects the micro-movements of a phone being placed or removed.
- Developing embedded firmware that processes sensor data in real-time and triggers motor-driven clamping mechanisms accordingly (clamping upon placement, releasing upon retrieval).
- Conducting user testing in simulated driving environments to evaluate the mount's responsiveness, mechanical reliability across different phone sizes, and impact on driver reaction times.
- Producing comprehensive technical documentation, including circuit schematics, CAD mechanical designs, embedded code, and user manuals.

Tasks & Deliverables

The selected group of students will form a software and hardware engineering team and work together to complete a full cycle of the software and engineering design process. Relevant stakeholders will be identified, and requirements engineering processes used to identify exact project requirements. A project proposal will be prepared, and the project will be implemented over the course of the GRP time period.

Because the team will need to go through requirements engineering, part of this project will involve applying an appropriate requirements elicitation methodology. The requirements engineering, feasibility analysis, and project scoping will all need to be completed early in the GRP time frame.

The team will need to make informed decisions about which engineering processes, hardware architectures, and software methodologies to apply to this project.

Over the course of the project, the team will need to produce several deliverables, which may include (but are not limited to):

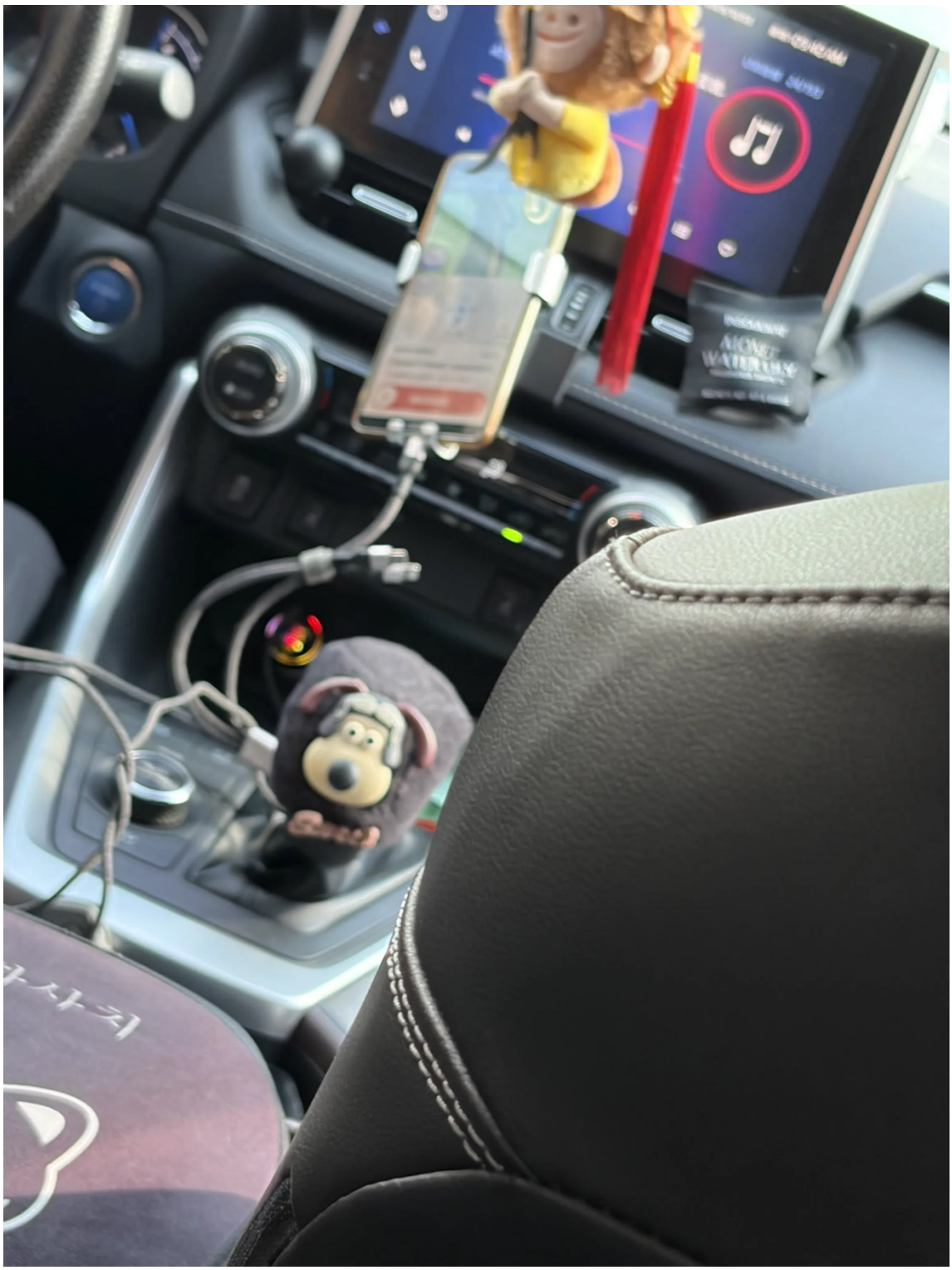
- a report on the current situation of car mounts and driver distraction
- a proposed system architecture combining hardware and software components
- a system requirements specification (SRS)

- mechanical design documentation (CAD files) and circuit schematics
- team management and planning documentation
- prototypes (breadboard, PCB, and 3D-printed mechanical housing)
- progress reports
- verification and validation plans (e.g., stress testing, battery drain tests)
- code and code documentation (embedded C/C++/Python, mobile companion app if applicable)
- tests and test reports (including usability and safety evaluations)
- instructions/user manual(s)

The target goal of the SE project will be to deliver, on time, a system that seamlessly integrates hardware sensing with software logic to meet the safety and convenience requirements of the various stakeholders. Please note that the project supervisor do not make hardware recommendations, but would financially support the (reasonable) acquisation of related hardware platform.

Additional Information

This photo is the initiation of the idea of this project – sorry for its resolution.



[P18] An Automated Testing Framework for Conversational Agentic Systems

Requirements

This project is suitable for computer science students interested in software testing, large language model (LLM) based applications and conversational agents, and distributed systems.

Students should be willing to undertake a substantial software engineering task involving automated testing, distributed task orchestration, and the evaluation of AI-driven systems. Familiarity with Python, Javascript, and basic software testing concepts would be beneficial.

Background

The growing adoption of agentic systems based on LLMs has introduced new challenges for software testing. Their complex, non-deterministic behaviour and diverse outputs make it difficult to define expected results, while exhaustive testing is often infeasible due to large input spaces and high execution costs.

Metamorphic testing alleviates the "test oracle problem", which refers to the difficulty in determining the correctness of the test results, by checking whether predefined relations (known as "metamorphic relations") among multiple executions hold. It is particularly useful for systems whose output is difficult to assess. By identifying behavioural inconsistencies without requiring an explicit expected output for each test case, metamorphic testing offers a practical approach to evaluating otherwise difficult-to-test systems, including LLM-based applications and conversational agents.

Another challenge is that large-scale testing is time-consuming, especially when running the test cases involves network communication, model inference, and sequential processing stages. Distributed execution can mitigate these bottlenecks by coordinating tasks across multiple machines, enabling greater concurrency and improved resource utilisation.

This project aims to integrate metamorphic testing with distributed test automation to facilitate the scalable and reliable evaluation of LLM-based conversational agents.

Outline

This project aims to design and implement a distributed testing framework for large-scale, long-running automated tests of LLM-based applications, particularly conversational agents. Building on the open-source Airtest framework where appropriate, the system will extend conventional metamorphic test automation through distributed task scheduling, concurrent execution, fault recovery, and scalable pipeline orchestration.

The framework will aim to support high-throughput test execution, operational reliability, and fault tolerance through distributed task scheduling, concurrent processing, and failure recovery. It will also support diverse user interactions and application scenarios, with centralised result management and detailed test reporting.

Testing conversational agents presents challenges due to network latency, model inference costs, and sequential processing bottlenecks. This project will investigate how distributing independent test tasks across networked machines can reduce end-to-end execution time while maintaining reliable coordination and result traceability under constrained computational resources.

Tasks & Deliverables

Expected tasks and deliverables include:

1. Design and implement a distributed testing platform. Build a distributed testing system around Airtest and related components, enabling sequential testing workflows to be decomposed into tasks or pipeline stages and dispatched across multiple networked machines. The platform should support concurrent execution, task-state tracking, failure handling, and reliable result collection.
2. Develop a proof-of-concept testing system for conversational agents. Integrate automated test execution and metamorphic testing into the distributed platform. Demonstrate that the system can execute test cases, collect the resulting outputs and produce evaluation results.
3. Evaluate the framework experimentally. Assess its performance and reliability under different workloads and levels of concurrency. Relevant metrics may include end-to-end execution time, task throughput, failure recovery, and scalability as additional workers are introduced.

The final deliverables are expected to include a working prototype, its supporting implementation and documentation, and an evaluation report presenting the experimental results.

Additional Information

This project involves a substantial engineering component and requires knowledge spanning software testing, distributed systems, and LLM/agent interactions. Students should review the project scope and the recommended resources before selecting it. Some of the requirements may change over the course of the project based on the industry partner's needs.

References

https://github.com/AirtestProject/Airtest The GitHub repository of the Airtest Project

https://airtest.doc.io.netease.com/en/for_newer/ An introduction to Airtest

https://arxiv.org/abs/2605.13898 Z. Zheng et al., “Bidirectional empowerment of metamorphic testing and large language models: A systematic survey,” 2026.

[P19] SmartCareer: A Personalized Employment Recommendation System with a Multi-Dimensional Job Quality Evaluation Model

Requirements

This project will require solid Python programming skills, working knowledge of machine learning and information retrieval fundamentals (feature engineering, ranking and evaluation metrics), and basic familiarity with natural language processing for extracting skills from job descriptions. Teams will also need database skills (SQLite/MySQL, optionally Neo4j), basic web development (a RESTful API and a simple front end), and good team-working habits including version control with Git. A genuine interest in labour-market analytics and working with large, real-world recruitment data is important.

Background

Artificial intelligence is reshaping the global labour market: it eliminates routine tasks, creates new occupations and shifts skill demand faster than graduates can adapt. The World Economic Forum's Future of Jobs Report (2025) predicts that 22% of global jobs will undergo fundamental change by 2030, and studies forecast that over one-third of jobs in China will be significantly affected by AI in the coming decades. As a result, graduates face a structural “high supply, low adaptation” problem: many cannot find jobs that genuinely fit their skills, interests and career expectations, while employers struggle to fill skill-specific vacancies.

Employment quality is multi-dimensional, covering not only salary and stability but also career-job match, growth potential and satisfaction. Existing platforms and simple recommendation methods rely on keyword matching or collaborative filtering, which suffer from cold-start and data-sparsity problems and cannot judge whether a job is actually “good” for a particular student.

This project addresses this gap. Using real recruitment data already collected by the research group, it will build software that evaluates jobs across four quality dimensions – salary level, growth space, stability and skill fit – and recommends the most suitable opportunities to individual students.

Outline

This project aims to design and build SmartCareer, a software system that provides personalised employment assistance to university students. The system comprises the following components:

Data processing module – ingests and cleans the already-available dataset (9.65 million graduate job postings, 2014–2025, six Chinese recruitment platforms, plus Zhejiang subsets), extracting job function, salary range, education and experience requirements, skills, industry and region.

New-occupation and skill-trend mining – applies time-series forecasting machine-learning models to the recruitment data to mine skill evolution trajectories and emerging job features, identify high-potential new occupations and their growth points, and build a graph of new jobs that graduates can enter.

Job quality evaluation model – scores every job on four dimensions: salary level, growth space, stability and skill fit, with transparent, tunable indicator weights so that decisions are explainable.

Student profile module – builds a profile of each student from their major, skills, qualifications, career preferences and priorities.

Recommendation engine – combines skill-based matching with the job-quality score and produces a personalised, ranked list of recommendations, each with a brief explanation.

Web application – a user-friendly interface for profile entry, ranked recommendations with explanations, and feedback that tunes the model weights.

Evaluation – measures recommendation quality with HR@k, NDCG, MRR and user trials, and delivers documentation, source code and a demo.

Tasks & Deliverables

The project is organised into six phases, each ending with a concrete deliverable:

Phase 1 – Data engineering. Clean and structure the provided dataset and build a data dictionary. Deliverable: a clean, queryable dataset with documentation.

Phase 2 – Feature extraction and trend mining. Extract skills with NLP, normalise salaries and build growth/stability indicators; apply time-series forecasting machine-learning models to mine skill evolution trajectories and emerging job features, identify high-potential new occupations and their growth points, and construct a graph of new jobs graduates can enter. Deliverable: feature vectors, a skill dictionary and the new-job graph.

Phase 3 – Evaluation model. Implement scoring for the four quality dimensions with adjustable weights. Deliverable: a documented, validated evaluation model.

Phase 4 – Recommendation algorithm. Implement skill-based matching and hybrid ranking fused with quality scoring (optional graph- or embedding-based extensions), evaluated with HR@k, NDCG and MRR. Deliverable: the algorithm module with an evaluation report.

Phase 5 – Web application. Build profile entry, recommendation display with explanations, and the feedback loop. Deliverable: a working web application.

Phase 6 – Integration and delivery. End-to-end testing, user trials, final report, code repository and demo. Deliverable: the complete system.

Additional Information

Data availability. The project needs no new data collection. The research group already holds 9,652,000 graduate job postings (2014 – June 2025) from six major platforms – 51job, BOSS, Zhilian, Liepin, Lagou and Kanzhun – updated in July 2025, including 512,550 Zhejiang graduate records and 684,735 Zhejiang listed-company records. Fields include company name, job function, salary range (monthly low/high), full job description, and education and experience requirements – sufficient to support all phases.

Suggested technology stack. Python (pandas, scikit-learn, optionally PyTorch); Flask/FastAPI back end; a simple front end (React, Vue or plain HTML/JavaScript); SQLite/MySQL storage; optionally Neo4j for graph modelling.

Team and roles. 3–5 members covering data engineering, model/algorithm development and web development, with regular progress meetings and version control with Git.

Deliverable format and supervision. Final submission: project report, source-code repository and live demo, assessed per the module's group-project rubric; supervisor confirmed at kick-off.

[P20] A Large Language Model-enabled Learning Pipeline with Genetic Optimizers for Generating Inorganic Materials as Texts

Requirements

This project primarily involves developing a Machine Learning system capable of generating inorganic materials represented as texts. Several key technologies and software tools are required and it is anticipated that the selected team will develop proficiency in all of them. They include Large Language Models (LLMs) as the key tool to generate inorganic materials represented in textual forms, Genetic Algorithms (Gas) and other Evolutionary Algorithms (EAs) that are fine-tuned to operate on strings to optimize generated inorganic materials in textual form, and various Chemical simulation tools developed within higher-level programming language frameworks (such as Python and MATLAB) for the purpose of simulation and verification of generated inorganic materials for the feasibility and actual stability.

A further potential requirement of this project is to document and develop educational resources around the key technologies and software tools used to develop the learning pipeline. There is a preference that the softwares and libraries used would be open-sourced and that the completed pipeline would be filed under the Open Source Initiative (OSI) framework. These materials would serve as part of onboarding materials for use by future Software Engineering Group Project (or as part of projects for other student research-based works). It is anticipated that iterative development of this learning pipeline be carried out over several Software Engineering Project cycles.

One key personnel requirement of the selected team is having an interest beyond typical Computer Science target applications; in this case interest in Chemical Engineering works. The team would be required to work with a domain expert and familiarize themselves with some domain knowledge involving inorganic chemistry.

Background

There is a need to develop Machine Learning assisted pipeline for the search and generation of novel but crucially stable inorganic materials (such as those having 3D crystalline structures) due to the very high costs required in actual physical and chemical testing of those materials. The real-world challenge surrounding the discovery of new materials are two-fold: (1) The number of known and stable materials form small datasets in most target applications of material science. (2) Each datapoint in a target dataset is typically high-dimension, involving composition, structure, various complex properties.

Machine Learning have been incorporated into these pipelines, with various new proposed approaches. The advent of LLMs has further accelerated the development of these pipelines to make

them more effective and efficient. However, a large number of these approaches are predictive in nature, where the LLMs for example serve as distribution models with which one can iteratively sample to generate the required symbolic sequences representing (in this problem domain) inorganic materials in crystal structures. GAs are powerful methods capable of manipulating and optimizing string-like data structures. Existing learning pipelines seldom incorporate an optimization cycle and it is anticipated that the use of GAs can improve those learning pipelines.

Outline

This project aims to develop a LLM-enabled learning pipeline with Genetic optimizers for the task of discovering metastable inorganic materials represented as texts. The learning system will be trained on standard datasets of the field, e.g., MP-20 dataset that consists of 45321 stable materials and from the Materials Project consisting of 120000 crystals with basic properties to enable comparison with the established state-of-the-art.

One possible key feature of this project would be the use of Open Source softwares and libraries with the anticipated that the developed software would be file under the framework of OSI.

Furthermore, it is anticipated that the software development phase and outcomes are documented in a manner consistent with the best practices of good software engineering. The produced materials would be used by future project students for further improvements.

Tasks & Deliverables

The stakeholder and domain expert (Chemical Engineering) has already been identified, together with a proposed learning pipeline. The two main challenges would involve acquiring some domain knowledge outside of Computer Science and Software Engineering, in this case Chemical Engineering and more specifically material science, and preferably producing a software that can be filed under OSI.

Phase 1: Requirements gathering, planning, and domain knowledge

- Going through and identifying key requirements of the project from the stakeholder.
- Outlining project development, timelines, and resources.
- Acquiring domain knowledge pertinent to the project.

Phase 2: System architecture and design, dataset search

- Familiarizing with the proposed pipeline from the stakeholder and proposed improvements (e.g., in terms of addressing open source requirement, newer libraries).
- Gathering appropriate datasets including newer ones based on more recent published research.

Phase 3: Software development

- Build the software to conform to the designed pipeline.
- Verify software performance both in terms of requirements meet and backend pipeline performance against existing state-of-the-art.

Phase 4: Documentation and OSI filing

- Documentation preparation for future use.
- Filing the produced software under OSI.

Main deliverables

- A complete report detailing the project (background, systems requirements, resource management, codes, testing, reflection, user manuals, etc).
- A complete software implementation of the LLM-enabled learning pipeline with Genetic Optimizers capable of generating inorganic materials in textual representation.
- (Preferably) OSI filing.

Additional Information

Key academic articles:

1. N Gruver, A Sriram, A Madotto, A Wilson, L Zitnick, and Z Ulissi, “Fine-Tuned Language Models Generate Stable Inorganic Materials as Text,” Workshop: AI for Accelerated Materials Design (AI4Mat-2023).
2. T Xie, X Fu, O-E Ganea, R Barzilay, and T Jaakkola, “Crystal Diffusion Variational Autoencoder for Periodic Material Generation,” International Conference on Learning Representations (ICLR-2022).
3. W Sun, ST Dacek, SP Ong, G Hautier, A Jain, WD Richards, AC Gamst, KA Persson, and G Ceder, “The Thermodynamic Scale of Inorganic Crystalline Metastability,” Science Advances 2, e1600225 (2016).

Short Guide to Crystallographic Information File (CIFs): https://www.ccdc.cam.ac.uk/community/access-deposit-structures/deposit-a-structure/guide-to-cifs/

Video presentation of article 1: https://neurips.cc/virtual/2023/78469

[P21] AI-Driven Post-Investment Quarterly Report Analysis for Private Equity Portfolios

Requirements

This project will require experience in natural language processing, document parsing (PDF/Word), large language models, financial indicator analysis, and data visualization. Familiarity with private equity post-investment workflows, portfolio monitoring, and industry benchmarking would be beneficial. The prospective team should be comfortable with Python, LLM-based information extraction, and time-series analysis.

Background

Private equity post-investment teams must regularly collect quarterly operational reports from numerous portfolio companies. Manually organizing, cross-period comparing, industry benchmarking, and identifying risks in these reports is extremely labor-intensive, and manual analysis often suffers from inconsistent standards, missing comparison dimensions, and delayed risk detection. There is a clear industry need for an AI system that ingests portfolio companies' quarterly reports and automatically produces standardized post-investment reports, completing dynamic analysis of both company-level and industry-level quarterly changes, while extending to benchmarking, risk control, and value-creation advisory outputs.

Outline

This project aims to develop an intelligent system for automatic post-investment quarterly report analysis. The core research content includes:

1. Non-standard document parsing and structured extraction - PDF/Word text and table extraction, structuring free text, financial indicators, and business descriptions into a database.
2. Dual dynamic analysis - company internal time-series (QoQ, YoY) with automatic anomaly attribution, combined with industry quarterly environment changes (policy, market sentiment, track shifts) to judge benefit/pressure.
3. Comparative risk and benchmarking engine - peer matching within the same track and stage, relative position and quarterly ranking movement, plus multi-dimensional risk warning (operational, compliance, team changes, performance-bet (VAM) completion) with risk grading.
4. Standardized report generation - an LLM-based module that assembles the above into a standardized post-investment report template, including hold/exit/impairment indications and value-creation suggestions as generated text.

Tasks & Deliverables

The tasks include designing the document parsing pipeline, developing the time-series and industry linkage analysis modules, building the benchmarking and risk warning engine, and implementing the standardized report generation module. Deliverables include:

1. A prototype intelligent parsing and post-investment report generation model.
2. A standardized post-investment report template adaptable to the firm's internal reporting.
3. A quarterly dynamic monitoring indicator system and risk warning rule base for portfolio companies.

Additional Information

Potential extensions include dynamic project value assessment using industry valuation multiples, a multi-quarter visualization dashboard, and longitudinal monitoring of the portfolio asset pool.

[P22] AI-driven Magnetic Nano-grain Analysis and Evaluation System

Requirements

This Software Engineering (SE) project will require to develop an AI-driven automatic analysis, 3D reconstruction and evaluation system for magnetic nano-grains.

Background

The current situation of analysis and evaluation for magnetic nano-grains is manually, which is time consuming and costs manpower, and also depends on the expertise and experience. Accordingly, there is an ever-increasing need to develop an automatic analysis, 3D reconstruction and evaluation software for magnetic nano-grains.

Outline

This SE project aims to utilize Computer Vision and AI techniques to analyse, reconstruct and evaluate magnetic nano-grains automatically. The students will form a software engineering team, and work together to complete a full cycle of the software engineering process, resulting in delivery of a tool.

The SE team will go through a complete requirements of SE process to identify the exact SE project requirements with the Ningbo Magnetic Company. Part of this will involve applying an appropriate requirements elicitation methodology.

The SE team will need to make informed decisions about which SE process approaches or methodologies will apply to this project.

Tasks & Deliverables

The prospective SE team will have familiarity with SE process. Over the course of this project, the SE team will need to produce several artefacts, including (but not limited to): a report on the current situation, a system requirements specification, design documentation, team management and planning documentation, prototypes, progress reports, verification and validation plans, code, code documentation, and instructions manual.

[P23] Automated Assessment Platform with Teacher-Confirmed Multimodal Feedback

Requirements

Automated marking of student coursework often gives a student a score and brief feedback, and little else. In this situation, a student can see that there is a mistake in their work, or an opportunity to improve the quality of their submission, but not always how to approach it differently in the future. The feedback itself might not be engaging or detailed enough to promote student learning. At the same time, a teacher with a large cohort cannot tailor feedback comments to every submission. Instead, feedback may be generic, or it arrives late, when students follow up in the lab or during office hours. Generative AI (GenAI) can help here. A model can read a submission and produce feedback that names the likely fault and suggests a fix, in a way that scales. Equally, however, a model can say something contrary to what the teacher is aiming to teach (a shortcut, for instance), reinforce bad habits, or, in the worst case, be simply wrong. Fundamental to the philosophy of the system, therefore, is that the teacher must lead the use of the model and approve what each student receives.

Background

An existing platform, developed by UNNC students, already provides some of this functionality. The system generates signatures, which match patterns in student submissions and have text feedback associated with them. When a student submission matches a given signature, the associated feedback is presented in the feedback report given to the student. A teacher is involved during this process, confirming the suitability of the signature and the feedback against the intended learning outcomes. However, students often overlook text feedback, or do not read it closely, and miss the learning opportunity. In addition to generating signatures and text feedback, GenAI can also produce media (a marked-up screenshot, a diagram, a short video demonstrating a fix, etc.).

The aim of this project, therefore, is to extend the existing system so that it can produce multimodal feedback, while keeping the teacher in control of the feedback a student sees.

Outline

The group will extend the platform to add signatures capable of communicating media as well as text in student feedback. Each item of media will be generated via GenAI with feedback (approve/deny/suggested changes) given by the teacher to inform subsequent revisions of the generated media.

Tasks & Deliverables

The group can expect to work on the following during the course of this project:

1. Extend signatures to support media alongside text;
2. Build a GenAI-driven media generation service;
3. Adapt the existing teacher review screen to support media-augmented signatures; and
4. Include media in the student report and feedback interface.

[P24] FitTwin: An AI Digital Twin for Personalized Strength Training and Recovery Optimization

Requirements

This project combines Artificial Intelligence (AI), Machine Learning (ML), sports analytics, human-centred AI, and software engineering. Students will gain experience in predictive modelling, recommender systems, data analytics, ML frameworks (e.g., PyTorch or TensorFlow), and modern web or mobile application development.

The project will involve collecting and analysing workout and fitness data from a variety of sources, including workout logs, wearable devices, and publicly available fitness datasets. Students will investigate how AI can model an individual's training behaviour and generate personalized recommendations for improving performance and recovery.

Particular emphasis will be placed on developing a digital twin of a gym user. This virtual representation should continually learn from training history, workout performance, and recovery patterns to predict future outcomes and support personalized decision-making.

The project is suitable for students interested in AI, data science, fitness technologies, sports performance, and human-centred application design. Prior experience with ML, data visualization, or web development would be beneficial, although not essential.

Background

Strength training is one of the fastest-growing forms of physical activity, attracting individuals aiming to improve athletic performance, increase muscle mass, enhance physical health, or simply maintain a healthy lifestyle. Despite the widespread availability of fitness applications and online training programmes, many individuals struggle to determine whether their training strategies are effective, whether they are recovering adequately, and whether they are progressing towards their goals.

Most commercial fitness applications rely on static programmes or simple heuristics that do not adapt to individual users. In practice, however, training outcomes are influenced by numerous factors, including workout volume, intensity, exercise selection, recovery quality, training consistency, and individual physiological responses. As a result, identical training programmes can produce significantly different outcomes for different individuals.

Recent advances in AI have created opportunities to develop intelligent fitness systems capable of continuously learning from user data and adapting recommendations over time. A promising concept is the digital twin, a virtual representation of a real-world entity that can be used to model behaviour, predict future outcomes, and support decision-making.

By applying digital twin concepts to strength training, it becomes possible to create a personalized model of an individual athlete that evolves as new training data become available. The system could forecast strength progression, estimate recovery needs, identify training plateaus, evaluate alternative workout strategies, and generate adaptive recommendations tailored to each user's goals and capabilities.

This project explores how AI-powered digital twins can be used to improve strength-training outcomes and provide users with more personalized, explainable, and data-driven coaching support.

Outline

The main goal of this project is to design and develop **FitTwin**, an AI-powered digital twin platform for personalized strength training and recovery optimization.

The proposed system will create a virtual representation of a user's training profile by analysing historical workout data, exercise performance, training volume, workout frequency, and recovery indicators. The digital twin will be continuously updated as new information becomes available, enabling the system to model an individual's current condition and predict future performance trajectories.

Using ML and predictive analytics, FitTwin will help users answer questions such as:

- How much strength improvement can be expected over the next several weeks?
- Is current training volume appropriate for achieving specific fitness goals?
- Has recovery been sufficient to support continued progression?
- Are there early signs of overtraining or performance stagnation?
- What training modifications are most likely to improve future outcomes?

An important feature of the platform will be its ability to simulate alternative training scenarios. For example, the digital twin may estimate how changes in training frequency, workout volume, exercise selection, or recovery strategies could affect future strength gains. This allows users to explore multiple training options before making real-world adjustments.

The project may investigate research questions such as:

- Can AI models accurately predict future strength progression from historical workout data?
- Can digital twin models improve the personalization of training recommendations?
- How effectively can ML estimate recovery readiness and fatigue levels?
- Can explainable AI techniques improve user understanding and trust in AI-generated coaching recommendations?

- How accurately can digital twins simulate the outcomes of alternative training strategies?

Students will be expected to design, implement, and evaluate a complete software solution comprising data acquisition, ML models, digital twin generation, recommendation engines, visualization components, and user interfaces.

The resulting system should provide users with personalized insights into their training progress while also serving as a platform for exploring emerging research challenges in AI-powered sports analytics, digital twins, and personalized recommendation systems.

Tasks & Deliverables

Students will work as a software engineering team to design, develop, and evaluate the FitTwin platform.

Potential tasks include:

- Reviewing literature on sports analytics, digital twins, and AI coaching systems
- Collecting and analysing workout and fitness datasets
- Developing ML models for strength and performance prediction
- Designing digital twin representations of users and training behaviours
- Implementing personalized recommendation algorithms
- Developing recovery and fatigue estimation mechanisms
- Building visualization dashboards for performance forecasting
- Investigating explainable AI methods for recommendation transparency
- Developing a web or mobile application for user interaction
- Conducting technical and user-centred evaluations

Deliverables may include:

- System architecture and implementation documentation
- Workout analytics and visualization modules
- Strength progression prediction models
- Digital twin engine
- Personalized recommendation engine
- Recovery and readiness estimation modules

- Explainability dashboard
- Web or mobile application
- Experimental evaluation report

Additional Information

This project has strong links to current research in AI, Sports Analytics, Digital Twins, Human-Centred AI, and Personalized Recommendation Systems

Strong teams may have opportunities to investigate publishable research topics such as:

- Digital twin modelling for human performance prediction
- Explainable AI for strength-training recommendations
- Recovery and readiness forecasting using ML
- Personalized adaptation of training programmes
- Human trust and adoption of AI-powered coaching systems

Potential extensions include:

- Injury-risk prediction and prevention
- AI-assisted exercise selection and workout design
- Nutrition-aware training recommendations
- Wearable-device integration
- Retrieval-augmented fitness coaching assistants
- Reinforcement learning for adaptive training optimisation
- Multi-user benchmarking and athlete comparison

The project is particularly suitable for students who wish to apply modern AI techniques to real-world fitness and sports-performance challenges while exploring one of the fastest-growing research areas at the intersection of AI, digital health, and personalized decision support.

[P25] AI Wardrobe Studio

Requirements

The team will design and build an interactive web application that helps users create outfits from clothes they already own. The project involves Python, web development, databases, image processing and integrating Qwen or comparable multimodal language models. Basic programming skills are required; the team will use pretrained models and existing libraries rather than train a large model.

Background

Finding an appealing outfit in a photograph is easy; recreating it with the clothes in your wardrobe is harder. An AI wardrobe assistant can connect visual inspiration with practical choices for everyday activities, presentations and social events. This project explores how image understanding, clothing retrieval and conversation can help users discover combinations and adjust them to their preferences.

Outline

The aim is to build a personal wardrobe studio where users upload clothing photos, request outfit suggestions and refine the results. For example, a user could upload an inspiration image and ask, "Can you recreate this look with my clothes?" They could then request, "Keep the trainers, but make the rest smarter."

- Upload and manage clothing photos. Use a multimodal model to suggest clothing categories, colours and styles, with editable labels and availability status. Produce clear clothing cards for browsing and outfit displays.
- Retrieve clothes that match a text request or inspiration image. Recommend combinations using existing wardrobe items, explain substitutions, and account for the occasion, user preferences and weather conditions entered by the user.
- Provide an interactive outfit board. Let users compare suggestions, keep selected items, replace individual pieces and refine combinations through conversation. Save favourite outfits and feedback.
- Connect model outputs to the application through structured data and controlled operations. Validate clothing references and constraints, ask for clarification when needed, and handle unavailable model services.

- Generate a virtual try-on preview for one upper-body garment using a pretrained model such as Leffa. Use a consented person photo and a selected wardrobe item; show the generated image as a visual preview rather than a prediction of physical fit.
- Use "Pack my weekend" to create outfits for a short trip within a user-selected item limit, generate a packing checklist, and swap garments while keeping the plan within that limit.
- Take a "Weekly outfit challenge" by creating a week of looks from a small selection of clothes. Highlight rarely worn items, compare favourite combinations, and track outfit variety and garment reuse.

Tasks & Deliverables

- Identify user needs, review related applications and agree a requirements specification, project scope and acceptance criteria with the supervisor.
- Design the interface, database and application architecture.
- Use version control, regular integration, code review and documented meetings. Maintain the project website and clear records of team contributions.
- Test the software at component and system levels. Compare the final recommendation workflow with a simple baseline on a fixed set of wardrobe tasks. Assess relevance, constraint satisfaction, invalid item references, response time and model usage costs; explain failures and design trade-offs.
- Benchmark try-on latency on the selected GPU before finalising performance requirements. Proposed target: 95% of requests display a result within 30 seconds from submission, including transfer, preprocessing, inference and rendering, with the model preloaded, one active request and a fixed supported resolution. Test at least 50 representative requests, report failures and cold-start times separately, and also test concurrent demand. This is a target to validate, not an established guarantee.
- Deliver the working application, documented source code, sample wardrobe, setup instructions, user guide and test report. Prepare the required interim and final reports, individual reflections and peer assessment, demonstration, presentation and promotional materials.
- Keep the try-on model loaded in a dedicated GPU worker, limit active jobs, and display queue status without blocking the interface. Cache reusable preprocessing. At a 60-second waiting limit, stop waiting and offer retry or the outfit board; a fallback does not count as a successful try-on result.

Additional Information

Qwen3-VL — multimodal understanding for garment descriptions and reference looks. https://github.com/QwenLM/Qwen3-VL

Wardrowbe — MIT wardrobe app with weather-aware suggestions and wear tracking; a reference for product and architecture design. https://github.com/Anyesh/wardrowbe

FashionCLIP — MIT repository for image-text clothing retrieval; a candidate for matching reference looks to wardrobe items. https://github.com/patrickjohncyh/fashion-clip

rembg — MIT background removal tool with Python and CPU support, for clean outfit boards. https://github.com/danielgatis/rembg

Leffa — primary try-on candidate. The official repository reports about 6 seconds per image on A100 with float16; this is not an end-to-end latency guarantee. Code repository:https://github.com/franciszzj/Leffa

FASHN VTON v1.5 — alternative open-source try-on implementation; no verified latency commitment in the reviewed README. Benchmark locally and check model and dependency licences:https://github.com/fashn-AI/fashn-vton-1.5

[P26] CodeLingo: Learning to Program Through Short Daily Games

Requirements

This project suits a team with some web development experience (for example Python with Flask, or JavaScript) and a basic database. Everyone should be comfortable with C or Java, since that is what the learners will practise.

Background

Many first-year students find programming hard, and a lot of them only practise just before a deadline or an exam. Apps for learning human languages have shown that short daily lessons, with streaks and points, get people to practise a little and often. Few apps do the same for programming, so this project builds one for our own students.

Outline

The aim is a web app, with its own name and design, where students do a few quick exercises each day and earn points and streaks. Exercises could include predicting what code prints, finding the bug, filling in a missing line, or putting scrambled lines in order, all checked automatically.

A simple adaptive feature will choose what to show next, for example repeating topics a student got wrong. Teams that finish early could try an AI model to suggest new exercises, as long as each one is checked before a learner sees it. A reasonable minimum is one language, about twenty exercises, user accounts, and a progress page.

Tasks & Deliverables

The selected students will form a software engineering (SE) team and complete a full cycle (maybe several) of the SE process. Stakeholders, such as students and programming lecturers, will be identified, and requirements engineering used to find out what the system needs to do. This, with feasibility analysis and scoping, should be finished early.

Deliverables may include (but are not limited to) a requirements specification, design documents, plans, progress reports, prototypes, code and tests, a user manual, and, if possible, a small trial with student volunteers. The target is a working system, delivered on time, that meets the stakeholders' needs.

Additional Information

The whole project can be built with free, open-source tools, and no paid services are needed. Start small: get one topic and one exercise type working end to end, using exercises that need no code execution. If learners later run their own code, tools such as Docker or Judge0 can do it safely in a sandbox. Do not copy the name, look or content of any existing learning app. A trial with students needs ethics approval first, and only essential data should be collected.

[P27] IndustrialSense: Software Platform for an Optical Sensing and Process Monitoring Prototype

Requirements

This project will suit a team with a strong interest in software engineering, systems integration, and real-world stakeholder work. Students should have good programming skills and be willing to learn about APIs, databases, user-interface development, data handling, and software testing. Experience with device communication, time-series data, embedded systems, or industrial software would be useful, but is not essential. The team should be comfortable working with incomplete requirements and evolving technical constraints.

Background

This project originates from a real industrial development request. An industrial partner is developing an engineering prototype that combines optical sensing components, supporting sensors, control hardware, and a supervisory computer to monitor an industrial process. The intention is to move from periodic or manually recorded measurements towards a more automated workflow in which data can be acquired, processed, stored, visualised, and used to support operational decisions.

Commercially sensitive details and final device specifications will not be fully disclosed at the start. Instead, the student team will work from a representative system concept and use requirements engineering to clarify the software scope with the project proposer and relevant stakeholders.

Outline

The aim is to design and implement the software platform around the prototype. The focus is on software engineering and integration rather than on the underlying chemistry, mechanical system, or detailed electronics.

The selected students will form an SE team and work together to complete a full cycle (maybe several full cycles) of the SE process. Relevant stakeholders will be identified, and requirements engineering processes used to identify the exact SE project requirements. A project proposal will be prepared, and the project implemented over the course of the GRP time period. Because the SE team will need to go through requirements engineering, part of this project will involve applying an appropriate requirements elicitation methodology. The requirements engineering, feasibility analysis, and project scoping will all need to be completed early in the GRP time frame.

Tasks & Deliverables

Over the course of the project, the SE team will produce a series of deliverables that form the basis of the project marking criteria. These include (but are not limited to):

- **Requirements engineering** — Identify stakeholders, elicit and prioritise requirements, and define the project scope. Produce functional and non-functional requirements along with acceptance criteria. *Expected deliverables:* a report summarising the state of the art in online industrial sensing technologies, and a well-structured system requirements specification (SRS).
- **System design** — Define a suitable system architecture, data model, device/interface abstraction, user interface, and key interaction or data-flow models. *Expected deliverables:* a proposed system design with supporting documentation.
- **Prototype implementation** — Deliver a working software system covering device/simulator integration, data acquisition, processing, persistence, monitoring, and operator interaction. *Expected deliverables:* source code with accompanying code documentation.
- **Verification and validation** — Conduct unit, integration, system, usability, and fault-scenario testing, providing evidence that the prototype satisfies the agreed requirements. *Expected deliverables:* a validation plan, test cases, and test reports.
- **Iterative stakeholder review** — Demonstrate progress at suitable milestones, collect stakeholder feedback, and refine the system and requirements as appropriate.
- **Project artefacts** — Throughout the project, maintain source code and code documentation, requirements and design documentation, project planning records, test/V&V documentation, setup and deployment instructions, and a concise user/technical manual.

[P28] Developing an Online Learning Platform for Competitive Software Testing and Bug Fixing

Requirements

This project involves web application development, software testing and software engineering education. The selected team will develop skills in web technologies, databases, version management and automated assessment, with Java and JUnit knowledge needed to support the submitted programs and tests. User interface design, access control and the isolated execution of submitted code will also be involved. The GRP team will design, implement and evaluate the platform; student teams using the platform will undertake the programming and testing activities it supports.

The GRP team may select suitable languages and frameworks for the frontend, backend and supporting services. Java is required for the student programs submitted for testing, and test cases must be written as JUnit tests. The platform must compile and execute these Java programs and JUnit tests in a consistent environment, using a shared program interface, an agreed JDK and JUnit configuration, controlled dependencies and a standard JUnit test template.

The team may also develop an open educational resource bringing together a sample activity, Java starter interfaces, test examples and guidance for teachers and students. This could support future adoption of the platform in software engineering teaching.

Background

AI-assisted programming motivates a reconsideration of coursework in which students find and fix predefined bugs in a fixed program. This project starts from the concern that such activities may provide fewer opportunities for students to demonstrate independent requirements interpretation, test design and iterative improvement. It proposes a practical environment in which these skills can be exercised together.

In the proposed learning activity, teams independently develop Java software from the same description. Students then design tests that challenge other teams' implementations, while improving their own software in response to failures. The test suite evolves through student contributions, creating a continuing cycle of development, testing, feedback and repair. A supporting platform must make this process fair, understandable and manageable for teachers, particularly when submitted tests are invalid, duplicated or potentially harmful.

Outline

The primary aim is to develop an online platform that enables teachers to organise competitive software testing and bug-fixing activities. The platform will support learning through requirements interpretation, peer testing, debugging and regression testing, while recording both testing contributions and software quality.

Key design and evaluation questions include:

- How can a shared software specification and Java interface support fair testing across independently developed implementations?
- How can test review and controlled execution admit useful edge cases while rejecting invalid or harmful submissions?
- How can scoring and feedback reward both the discovery of defects and the subsequent improvement of software?

The project will develop and evaluate a working prototype. Expected development activities include:

- Providing teacher facilities to publish software descriptions, interface contracts, activity rules and deadlines, alongside student accounts, team membership and submission management.
- Supporting versioned Java program submissions and individual JUnit test submissions credited to the student's team. Each test should state the requirement it checks, its expected result or assertion, and its purpose.
- Implementing a test approval workflow. Automated format and compilation checks will support teacher or teaching-assistant review of specification compliance, expected results, duplication and prohibited behaviour. Rejected tests should receive an explanation. Valid boundary cases should remain eligible.
- Executing approved tests against eligible teams' software in an isolated environment with time, memory and access limits. Both programs and tests must be treated as untrusted. Reports should distinguish software failures, invalid submissions and platform errors, and identify the versions tested.
- Allowing teams to upload repaired versions and rerun approved tests. Dashboards should present diagnostic feedback, test outcomes, version history and scores. Test submission should close before the final software deadline to provide a defined repair period.
- Evaluating the complete workflow through a sample activity, simulated team submissions and representative user feedback, including the clarity of review decisions and scoring.

Scoring will have two components. The testing contribution score will count how many opposing teams are affected by each approved test, counting each test-team pair only once and preventing duplicate tests from earning repeated credit. The software quality component will measure the approved tests that a team's final version fails, with fewer failures producing a better result. Earlier discovery credit will remain when another team repairs its software. The exact weights, applicable test set and treatment of exceptional cases will be agreed during requirements analysis.

Tasks & Deliverables

The selected students will form a software engineering team and work through a complete development lifecycle, with iterations as appropriate. They will identify stakeholders, including teachers, teaching assistants and student users, and apply suitable requirements elicitation methods. Requirements analysis, feasibility assessment and project scoping should be completed early enough to guide a realistic project proposal.

The team will make informed decisions about architecture, development methods, testing and project management. Particular attention should be given to the interface between the web platform and the execution service, the traceability of results to submitted versions, and the balance between automated checks and human review.

Expected deliverables include:

- A review of relevant teaching workflows and existing assessment tools, and a scoped project proposal.
- A system requirements specification, including user roles, Java submission contracts, review rules, scoring and acceptance criteria.
- System, database and interface designs, with an execution-isolation and deployment plan.
- Team management, iteration planning and progress documentation, supported by working prototypes.
- A working platform, source code and code documentation covering the complete development, testing and repair workflow.
- Verification and validation plans, automated tests and test reports, including scoring, permissions and execution limits.
- A demonstration activity with a shared description, Java starter interface, JUnit test template, several team implementations and repaired versions.
- Teacher and student user guides, deployment instructions, an evaluation report and recommendations for future development.

The target outcome is a demonstrable platform that meets the agreed stakeholder requirements within the GRP period. Evaluation should establish functional correctness, reproducible results and usability. Any claims about improved learning would require appropriate evidence beyond a successful technical demonstration.

Additional Information

The initial scope should remain suitable for a third-year software engineering team: one execution environment for submitted Java programs and JUnit tests, one standard submission interface and a small classroom deployment. This execution requirement does not constrain the platform's development languages or frameworks. A sample task such as a booking service or inventory component could provide meaningful functional and boundary tests without requiring complex application deployment. Support for submitted programs in other languages, comprehensive automatic test validation and AI features are optional extensions.

Useful starting points include:

- Discussing how teachers currently assess testing and debugging, and identifying the feedback students need to make effective repairs.
- Prototyping the smallest complete workflow: one Java submission, one reviewed test, an execution result and a corrected version.
- Agreeing how ambiguous requirements, equivalent tests, late submissions and disputed results will be handled before implementing detailed scoring.

Here, attack means exposing specification violations through valid tests, and defence means repairing the software. Attacks on the platform infrastructure are outside the learning activity. Future work could explore learning analytics, richer feedback or classroom studies, after the core platform is reliable.

References

JUnit project documentation and testing framework overview

Docker documentation on container resource constraints

[P29] Real-Time Surgical Video AI Assistant Development

Requirements

Prior experience in AI or full-stack development is not required; the necessary skills can be learned during the project with guidance from the client's research team. We are mainly looking for team members who have:

- Genuine interest in artificial intelligence, Vision-Language Models (VLMs), and their applications in healthcare
- Self-motivation and the ability to learn new tools independently (e.g., reading documentation and open-source code)
- A conscientious and responsible attitude towards teamwork, deadlines, and code quality
- Willingness to communicate regularly with teammates and the client's research team
- Basic programming skills (e.g., Python, Java, or web development), or other relevant abilities such as UI design

Background

Minimally invasive surgery is performed under continuous endoscopic video, which makes the surgical video an ideal input for intelligent intraoperative assistance. An AI assistant that watches the operation as it happens could answer questions from the surgical team at any moment, remember what happened earlier in a procedure lasting several hours, and proactively alert the team when a safety-relevant event occurs. A concrete example is the prevention of retained surgical items: sponges, needles, and clips left inside the patient remain a recognised never event, yet they are still tracked by error-prone manual counting. Most existing Vision-Language Models (VLMs), however, work offline: they analyse a complete video after the fact, and their memory cost and latency grow with video length. Streaming video large language models, such as the recently released open-source OneStreamer, overcome this limitation by processing frames as they arrive, keeping a compact memory of past events, and deciding by themselves when to respond. Meanwhile, the ORena FOCUS challenge (MICCAI 2026) has released expert-annotated question-answer data on full-length laparoscopic surgeries focusing on foreign objects. Combining the two into a working real-time system will turn recent research progress into a tangible tool for surgical education and future clinical evaluation.

Outline

This project aims to develop a real-time surgical video AI assistant platform. The team will fine-tune an open-source streaming video large language model (OneStreamer) on the ORena FOCUS surgical dataset, wrap the model into a streaming inference service, and build an interactive web interface that plays a surgical video as a live stream, allows users to ask questions at any time, displays proactive alerts, and visualises a timeline of what the model has observed. The system will be designed to be model-agnostic and data-agnostic, so that improved streaming models and new surgical datasets can be integrated with minimal changes.

Tasks & Deliverables

1. Data Preparation & Conversion

- Dataset Access: Download the ORena FOCUS data with the official orena-focus Python toolkit, including HeiCo-FOCUS-VQA (30 full-length laparoscopic colorectal procedures, about 96 hours of video, and 15,000 expert-validated question-answer pairs with timestamps) and, if access is granted, LapChole-FOCUS-VQA (170 laparoscopic cholecystectomy videos).
- Streaming Format Conversion:
  - Resample videos to 1 frame per second and align them with the question-answer timestamps
  - Convert each question-answer record into a streaming dialogue sample in the OneStreamer training format, with questions asked both before the evidence appears (proactive) and after it has left the recent video window (memory)
  - Keep the official train/test split and hold out a validation subset by video
- Data Interface: Implement the converter as a stand-alone module with a documented JSONL format, so that new surgical datasets only require a new converter.

2. Model Fine-tuning & Evaluation

- Baseline Setup: Deploy the released OneStreamer-4B model and reproduce zero-shot streaming inference on sample surgical videos.
- Fine-tuning: Apply LoRA-based supervised fine-tuning (SFT) on the converted surgical data, mixed with a small portion of general streaming data to avoid forgetting; if official training scripts are unavailable, use an open-source framework (e.g., ms-swift or LLaMA-Factory). At least one GPU with 48 GB memory is required.

- Evaluation: Compare the zero-shot and fine-tuned models on the test videos with the official orena-focus evaluation tools, reporting accuracy per capability group (e.g., recognition, temporal grounding, counting) and response latency.

3. Back-end Development (Python)

- Streaming Inference Service: A FastAPI and WebSocket service that ingests video (an uploaded recording played back in real time, or a live capture source), samples frames at 1 frame per second, and runs the model step by step, keeping per-step processing below one second through caching and mixed-precision inference.
- Unified Model Interface: An abstract streaming model interface (reset, push frame, add query, step), with OneStreamer implemented as one adapter selected through a configuration file.
- Scheduled Queries: Configurable recurring queries (e.g., foreign object counts every 30 seconds) whose answers feed the dashboard.
- Session & Event Management: Store all timestamped questions, responses, alerts, and model observations of each session in a database (e.g., SQLite or PostgreSQL).

4. Front-end Development (Web Platform)

- User Interface Design: Develop a professional interface suited to operating room displays (e.g., dark theme and large-screen layout), compatible with desktop and large monitors.
- Interactive Implementation:
  - Surgical video upload and real-time playback as a simulated live stream
  - Live chat panel for asking questions at any moment during playback
  - Proactive alert panel showing model-initiated messages with timestamps
  - Interactive event timeline of model observations, clickable to jump to the corresponding video moment
  - Foreign object status panel summarising the inserted and removed surgical items
  - Session report export (timestamped log in PDF or CSV)
- Technology Stack: React with TypeScript, WebSocket client, HTML5 video, and a component library (e.g., Ant Design or MUI).

5. System Testing

- Functional Testing: Verify all functional modules operate correctly
- Performance Testing: Measure per-step latency, time to first response, and GPU memory on full-length videos of two hours or more, and verify long-running stability

- Accuracy Testing: Validate model answers against expert annotations with the official evaluation metrics
- Usability Testing: Conduct a user study with at least five participants (e.g., medical students or clinicians) using the System Usability Scale (SUS)

6. Documentation & Deployment

- Technical Documentation:
  - Craft detailed system architecture design documents
  - Create user operation manuals and API documentation
  - Develop a model and data integration guide explaining how to add a new streaming model adapter or a new dataset converter
- Deployment Strategy:
  - Package the system with Docker Compose for deployment on a local GPU server, fully operational without internet access
  - Implement HTTPS and secure authentication mechanisms
  - Release model weights, configurations, and scripts for reproducibility, together with a recorded demonstration video

Additional Information

Upon completing this project, team members will have gained:

- Hands-on experience with modern deep learning frameworks (PyTorch) and Vision-Language Models (VLMs), including fine-tuning a large model with LoRA
- Practical skills in video processing and real-time data streaming (e.g., FFmpeg, OpenCV, WebSocket)
- Python back-end development and API design capabilities
- Modern web front-end development and UI/UX design experience
- Experience with Linux GPU servers, Docker deployment, and Human-Computer Interaction (HCI) system design
- Exposure to cutting-edge surgical AI research and the experience of turning a research model into a working software product

Regular project meetings and peer reviews will keep the project on track, and the client's research team will provide technical guidance and starter materials throughout.

[P30] “From Prompt to Film” — An AI-Empowered Micro-Film Studio

Requirements

This project requires a solid understanding of generative AI, particularly large language models and text-to-video diffusion models. Hands-on experience with PyTorch, Hugging Face Transformers, and OpenCV is strongly preferred. Familiarity with computer graphics, digital audio processing, and basic film editing concepts will be beneficial. The team should be comfortable working with high-performance computing environments (GPUs) and have some software engineering experience. An appreciation for storytelling and visual aesthetics is a plus, though a tolerance for occasionally bizarre AI-generated artifacts is mandatory.

Background

Micro-films are short videos ranging from 1 to 10 minutes and have become a key medium for storytelling, marketing, and entertainment. A popular example is the ”Three Lives, Three Worlds” wedding micro-film, which reimagines a couple's love story through the epic Chinese fantasy *Eternal Love*. These films often feature ancient costumes, magical realms, reincarnation, and vows that transcend lifetimes. However, traditional production is expensive and labour-intensive, involving scriptwriting, casting, filming, and post-production that can take months and cost tens of thousands of dollars. Recent advances in generative AI allow large language models to write fantasy romance narratives, text-to-video models to create scenes of immortal heroes and peach blossom forests, and voice and music models to produce complete Mandarin soundtracks. Despite this, automating a complete watchable micro-film remains an open challenge. This project builds an end-to-end system that turns a simple story description, such as a couple's love story with a ”Three Lives, Three Worlds” theme, into a finished wedding micro-film with minimal human intervention.

Outline

This project aims to develop a pipeline-based application that transforms textual narratives into playable micro-films. The core modules include:

- **Script Generation**: A fine-tuned large language model converts a high-level synopsis into a structured screenplay with scenes, character actions, and dialogue. It supports multiple genres, including romance, sci-fi, fantasy, thriller, and wuxia-style reincarnation epics.
- **Visual Scene Synthesis**: Text-to-video diffusion models generate short clips for each scene. Techniques such as ControlNet, LoRA, and temporal attention layers ensure visual coherence and stylistic consistency.

- **Character Consistency**: Face embedding models and reference-image conditioning maintain character identity across shots and camera angles.
- **Audio Generation**: The system synthesises Mandarin or English voiceovers with expressive tonal control and generates background music matched to the emotional arc of the story.
- **Automated Editing and Post-Processing**: A rule-based and AI-assisted module assembles clips, adds transitions, subtitles, and visual effects, and performs audio mixing to produce the final video file.

The system is delivered as a web application or desktop tool. The user inputs a story premise, selects a genre and style, and receives a downloadable MP4 file with embedded subtitles.

Tasks & Deliverables

The prospective team should be familiar with Python-based development environments and version control workflows. Key tasks and deliverables include:

- **Requirements Specification**: functional and non-functional requirements, user stories, and system architecture diagrams.
- **Dataset & Preprocessing Pipeline**: curated text-video pairs, character reference images, and annotation scripts for training and evaluation.
- **Trained Models & Evaluation Reports**: model weights and architecture descriptions for script generation, video synthesis, character consistency, and audio modules, benchmarked using CLIP score, FID, and user preference ratings.
- **Core Software Implementation**: API services, frontend UI, and integration modules unified into a single workflow.
- **Test Reports**: accuracy, inference latency, robustness, and failure-case analysis across diverse story prompts.
- **Deployable Application**: containerised service or cloud deployment with GPU support, along with installation and usage documentation.
- **Documentation Kit**: installation guide, API reference, prompt-engineering guidelines, and compliance notes including AI-generated content disclosure.
- **Maintenance Plan**: model update schedules, content quality monitoring, and fallback strategies for model failures.

Key success factors:

- The pipeline runs end-to-end with minimal manual intervention, from text prompt to video output.
- Generated videos are visually coherent, narratively aligned, and technically playable.
- The system generalises to out-of-domain prompts and produces intelligible output for unseen genres.

The project concludes with a live demo where users can input their own story ideas and observe real-time micro-film generation.

Additional Information

The use of AI in creative media raises ethical concerns regarding copyright, misinformation, and artistic authenticity. This project will follow responsible AI guidelines, clearly watermarking generated content as synthetic and ensuring that all training data is properly licensed. Relevant code and research papers on generative models and video synthesis can be found at the following resources:

- https://paperswithcode.com/
- https://github.com/

The system will remain genre-agnostic, capable of producing everything from a romantic reunion to a five-minute sci-fi mystery. Showcase demos may draw inspiration from popular fantasy dramas such as *Three Lives, Three Worlds, Ten Miles of Peach Blossoms*. Occasional unexpected AI behaviors are possible, but the pipeline will include fallback strategies to ensure reliable output.

[P31] Agentic Image Enhancement System for Challenging Visual Conditions

Requirements

This project will require the following skills and interests:

- Proficiency in programming, especially Python
- Interest in image processing, AI agents, and web-based user interface development
- Willingness to learn and integrate existing computer vision tools

Background

Images captured under poor lighting or affected by noise and blur can be difficult for people and computer vision systems to use. Different image enhancement techniques work well under different conditions, so a fixed processing pipeline may not produce satisfactory results for every image. This project explores whether an agent can inspect an image, select from existing enhancement tools, and evaluate the outcome to guide subsequent processing.

Outline

This project aims to develop a web-based system that allows users to upload an image, view its quality issues, and compare original and enhanced results. An agent will select and sequence suitable existing tools for a limited set of problems, such as low light and image noise. It will assess the processed image and, where appropriate, revise its choice of tools or parameters. The system will record the processing steps and allow users to accept a result or manually select an alternative method. A fixed enhancement pipeline will serve as a baseline for comparison.

Tasks & Deliverables

Tasks to be accomplished include: 1) overall system design; 2) integration of existing image enhancement methods; 3) design and implementation of the agent's tool-selection and evaluation workflow; 4) web-based user interface development; and 5) functional testing and experimental evaluation. The deliverable will be a working image enhancement system, together with documented comparisons between agent-selected processing and a fixed baseline.

Additional Information

The project focuses on integrating and evaluating existing techniques rather than developing a new image enhancement model. A minimum viable system may support only low-light and noisy still images; video processing and additional degradation types are optional extensions. On paired test data, students may report PSNR and SSIM alongside processing time. Images without reference originals should be assessed using appropriate no-reference or human evaluation rather than reference-based metrics. An LLM API is optional: the core workflow should remain demonstrable without a paid service.

[P32] RapidAid: Dispatching and Routing Medical Responders in an 80,000-Seat Stadium

Requirements

This project suits a team that wants to build a real, working system around a real algorithmic problem. Useful skills (or a willingness to learn them):

- Full-stack web development, database and API design, and real-time updates to a web client.
- Graph algorithms and optimization. Second-year algorithms knowledge is enough to start.
- Simulation, testing, UI/UX design, and team development with Git.

No medical or computer-vision background is required.

Background

It is the 70th minute of a sold-out match, and a fan collapses in the upper tier. A surveillance camera flags a ”person down” alert. The control room now has seconds to decide: which first-aid team should go, which route is fastest when the stairs are packed, where the nearest defibrillator (AED) is, and which part of the stadium will be left uncovered. For a cardiac arrest, every minute counts.

Ningbo is building a new stadium with around 80,000 seats. Because it is still under construction, the team will use it as an **imaginary pilot site**, build its own simplified model of it, and stage a football match or a concert there.

Outline

The goal is a **prototype dispatch-and-routing system for the stadium control room**. We assume live cameras with existing video analytics that report suspected incidents and crowd density. The team does not build the vision part. It uses a **simulated camera feed** and focuses on turning alerts into fast, sensible, traceable responses.

The stadium is modeled as a multi-level graph (seating blocks, concourses, stairs, lifts, gates, first-aid rooms, AEDs, ambulance bays) with walking times that change as crowds move.

Given the current incidents and responders, the system must:

- **Dispatch** the right responder(s), e.g., a first-aider plus an AED runner for a suspected cardiac arrest.
- **Route** them to the patient, and then route the patient out to a first-aid room or ambulance bay.

- **Adapt** when a corridor is blocked, a crowd surge occurs, or a more urgent incident arrives.

The team should build up in levels and compare them in simulation: (1) nearest responder by shortest path; (2) routing with live crowd and blocked-route data; (3) handling several incidents at once; and, as a stretch goal, (4) keeping every zone of the stadium within reach of help.

The system advises, but a human decides: an operator sees ranked options with a short explanation, then confirms or overrides them, and every decision is logged.

Tasks & Deliverables

The team will follow a full software-engineering lifecycle:

- Requirements and domain modeling (stakeholders: control-room operator, responder, venue safety manager).
- A configurable stadium model and a simulator for incidents, camera alerts (including false alarms), crowds, and blocked routes.
- A dispatch and routing engine with swappable strategies.
- A control-room dashboard (map, incidents, responders, routes, task status, audit log) and a simple mobile view for responders.
- Testing, plus a simulation study comparing strategies on measures such as time to first responder and time to AED.

Deliverables include the requirements and design documents, project plans and progress reports, the code repository with tests, an evaluation report, a user guide, and a final demo.

Additional Information

All data are synthetic. Medical diagnosis, facial recognition, and links to real emergency services are out of scope. A simple, well-tested, well-measured solution is valued over a complex one. Possible stretch goals include planning where to station teams before the event, replaying incidents for post-event review, and plugging in an off-the-shelf fall detector on public sample videos.

[P33] LLM-Empowered User Profiling for Recommendation

Requirements

• Proficient in programming with Python/Java

• Familiar with database systems

• Understand the working mechanisms of large language models

• Have a strong team spirit and be willing to devote to the group project

Background

Modern recommendation systems rely heavily on accurate user representations to deliver personalized experiences. Traditional user profiling approaches—such as collaborative filtering embeddings or hand-crafted demographic features—often produce opaque vectors that are difficult to interpret and inflexible across different recommendation scenarios. Large Language Models (LLMs) offer a promising alternative: they can transform raw user interaction histories into readable, semantically meaningful profile descriptions that capture preferences, behavioral patterns, and intent. However, deploying LLM-based profiling in practice is not simply a matter of calling an API. The generated profiles may be inconsistent, overly generic, or poorly aligned with downstream recommendation performance. Building a practical system that generates, evaluates, and refines LLM-based user profiles under realistic resource constraints requires careful engineering and hands-on experimentation. This project focuses on inference-only usage of existing LLMs, exploring how far prompt engineering, sampling strategies, and lightweight post-processing can go in producing high-quality user profiles for recommendation. The goal is to develop a usable software system that integrates existing LLM services into a complete profiling pipeline, with built-in evaluation and refinement mechanisms that do not require model training.

Outline

This project aims to develop an LLM-empowered user profiling system for recommendation, including a profile generation pipeline based on inference-only LLM usage, an evaluation and visualization interface, and a feedback-driven refinement mechanism. The system will be validated through integration with a standard recommendation task using a public dataset.

Tasks & Deliverables

1. Develop a basic user profiling pipeline that accepts raw user-item interaction histories and generates natural language profile descriptions by calling existing LLM APIs (e.g., GPT-series, Claude, or open-source models served via Ollama/vLLM) in inference mode only.
2. Design and implement a profile quality assessment module that measures generated profiles along dimensions including coverage, specificity, and relevance to downstream recommendation performance, without requiring any model training.
3. Design an interactive visualization interface that allows users to inspect generated profiles, compare profiles across different users, and examine the relationship between profile content and recommendation outcomes.
4. Implement a prompt engineering and sampling strategy component that explores how different prompt templates, temperature settings, and multi-sample aggregation affect profile quality, establishing practical guidelines for inference-only LLM profiling.
5. Brainstorm and implement additional lightweight profile enhancement mechanisms (e.g., template-guided generation for specific domains, cross-user profile comparison views, or rule-based post-processing to remove generic statements).
6. Test the developed system on a real-world recommendation dataset (e.g., MovieLens or Amazon Reviews) and evaluate whether LLM-generated profiles improve recommendation accuracy compared to baseline user representations, using only inference-time LLM calls.

Additional Information

[1] Wang, Lu, Di Zhang, Fangkai Yang, Pu Zhao, Jianfeng Liu, Yuefeng Zhan, Hao Sun et al. ”LettinGo: Explore user profile generation for recommendation system.” In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, v.2, pp. 2985-2995. 2025.

[2] Wu, Likang, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin et al. ”A survey on large language models for recommendation.” World Wide Web 27, no. 5 (2024): 60.